ORIGINAL ARTICLE

A benchmark study of optimizers for short-term solar PV power forecasting using neural networks under real-world constraints

Saloni Dhingra[1] · Giambattista Gruosso[1] · Giancarlo Storti Gajani[1]

Received: 10 April 2025 / Accepted: 23 July 2025 / Published online: 7 September 2025
© The Author(s) 2025

Abstract
Accurate short-term photovoltaic (PV) power forecasts are critical for efficient grid balancing, yet training optimizers, often overlooked compared to neural network architectures, significantly influence prediction accuracy and convergence speed. Prior research primarily focuses on adjusting network architectures, typically employing a single optimizer (commonly Adam), thus leaving optimizer selection underexplored, especially under noisy and incomplete real-world PV data. This study systematically benchmarks four optimizers—Adam, Adaptive Gradient (Adagrad), Rectified Adam (RAdam), and Lookahead—across three deep-learning architectures (Long Short-Term Memory (LSTM), Convolutional Neural Network (CNN)-LSTM, and LSTM-Autoencoder) using data from two distinct PV sites. Unlike prior works, we assess optimizer effectiveness across a wide range of conditions, including varying training data lengths, sampling intervals, and missing data patterns (both random and block-wise). Using two real-world PV datasets representing semi-arid and desert climates, we analyze forecasting accuracy, convergence time, and robustness. Our empirical results demonstrate that RAdam consistently outperforms Adam by achieving up to 36% lower forecasting error under noisy and incomplete data conditions, while Lookahead offers up to 40% faster convergence in deep hybrid models. These gains translate into tighter reserve-margin planning and smoother inverter set-points, advancing state-of-the-art PV forecast pipelines. The paper concludes with optimizer-architecture recommendations for practitioners facing latency or compute constraints.

Keywords Optimization · Time-series forecasting · Photovoltaic · Renewable energy · Convergence time · Hybrid models

1 Introduction

The increasing penetration of renewable energy sources, particularly solar PV systems, into the power grid requires accurate forecasting to ensure grid stability and efficient energy management [1]. However, the inherent intermittency and variability of solar power pose substantial challenges to reliable grid integration and efficient energy distribution [2]. Thus, accurate forecasting of solar PV power generation is critical to ensure optimal power system operation, enhance energy storage strategies, and support the wider adoption of renewable energy technologies.

Machine learning models have emerged as a powerful tool for tackling the complexities of PV power forecasting, particularly those that excel in time-series analysis [3]. Different neural networks, such as LSTM, CNN, and hybrid models, demonstrate strong capabilities in modeling the temporal dynamics and nonlinearities

Saloni Dhingra, Giambattista Gruosso and Giancarlo Storti Gajani have contributed equally to this work.

inherent in solar PV data [4–6]. Paper [7] proposes a novel methodology for deterministic wind and solar energy generation forecasting for multiple generation sites, utilizing multi-location weather forecasts. It employs a U-shaped temporal convolutional auto-encoder (UTCAE) architecture for temporal processing of weather-related and energy-related time-series across each site.

While these neural networks inherently possess the architecture needed to model complex time-series data, their performance is significantly influenced by the optimization algorithms used during the training phase. Optimizers are critical components that control the weight update process, directly affecting convergence speed, model stability, and predictive accuracy [8]. Among the many optimizers available, Adam, RMSprop, Stochastic Gradient Descent, and Adagrad are widely adopted for training deep learning models, each offering distinct advantages and trade-offs. However, the comparative impact of different optimizers on neural networks for PV power forecasting remains underexplored in current literature.

To address this gap, this study systematically evaluates the performance of various optimization algorithms across multiple hybrid neural network architectures, including CNN-LSTM, LSTM, and LSTM-Autoencoder (AE) models. We investigate how each optimizer influences forecasting performance across varying historical data spans, sampling intervals, and data integrity conditions. By conducting extensive experiments using two real-world PV datasets, we identify optimal optimizer-architecture combinations and draw insights on model generalization, convergence efficiency, and resilience to incomplete data. This work aims to provide a practical guide for enhancing short-term PV power forecasting through informed optimizer selection.

Stochastic optimization algorithms form the foundation of deep learning model training. The widely used Adam optimizer [9] introduced adaptive moment estimation, while extensions such as Adaptive Gradient (AdaGrad) [10], Root Mean Square Propagation (RMSprop) [11], and Adaptive Delta (AdaDelta) [12] focused on per-parameter learning rate adjustment. Recent innovations like Lookahead [13], Rectified Adam (RAdam) [14], Adaptive Belief (AdaBelief) [15], and Adaptive Gradient Clipping (AGC) [16] address convergence instability, gradient noise, and adaptiveness for deep architectures. Empirical studies [17–19] have demonstrated that the choice of optimizer significantly impacts model generalization and convergence in deep neural networks, especially for time-series and noisy input domains.

Several works have explored optimizer influence in time-series forecasting, particularly in energy and environmental systems. For instance, [4, 5, 20] investigate LSTM-based PV power forecasting, but adopt fixed optimizers (e.g., Adam or Nadam [21]) without a comparative evaluation. Others [22–24] combine evolutionary or nature-inspired optimizers with Bi-LSTM and CNN architectures, though they often focus on model enhancements rather than optimizer performance analysis. A comparative analysis of optimizers for weather and load forecasting is provided in [25, 26], emphasizing the growing need for standardized benchmarking across tasks.

The growing availability of PV datasets such as National Renewable Energy Laboratory Photovoltaic Data Acquisition (NREL PVDAQ) [27] has led to widespread use of LSTM, CNN–LSTM, and Autoencoder hybrids for solar forecasting [2, 8, 28]. While hybrid models have shown improved performance under complex irradiance dynamics, the role of the optimizer in handling data uncertainty, missing values, and varying temporal resolutions remains underexplored. Recent efforts [21, 29, 30] utilize advanced deep models but rarely isolate the optimizer's effect, motivating our study's focus on this critical training component. Table 1 offers a structured synopsis of the key solar-PV power-forecasting studies and explicitly flags the specific limitations or research gaps each one leaves unaddressed, thereby framing the open questions our work tackles.

A synthesis of the studies in Table 1 reveals five open issues:

1. Most works adopt a single default optimizer (e.g., Adam or Nadam) without systematic comparison across alternatives.
2. Comparative optimizer studies seldom include deep hybrids such as CNN–LSTM or LSTM-Autoencoder combinations.

Table 1 Representative literature on optimizer-driven solar-PV power forecasting and their unmet limitations

Study	Model	Optimizer(s)	Data res.	Reported limitation/gap
Wentz et al. [5]	LSTM	Nadam	1 min	Fixed optimizer; no missing-data analysis.
Symeonidis et al. [7]	UTCAE	Adam	60 min	Multi-site focus but no optimizer comparison.
Ahmed et al. [8]	CNN–LSTM	Adam	5 min	Does not test other optimizers; assumes complete data.
Zhou et al. [20]	Bi-LSTM + PSO	PSO tuned	10 min	Evolutionary optimizer explored, yet no baseline with Adam/RMSprop; missing-data untreated.
Sharma et al. [21]	CNN-GRU	Nadam	5 min	Reports accuracy only; lacks convergence analysis.
Li et al. [22]	CNN–LSTM	Differential Evolution	5 min	Meta-heuristic optimizer; evaluates a single climate site and offers no optimizer benchmark.
Peng et al. [30]	Bi-LSTM + GA	GA tuned	15 min	Uses nature-inspired optimizer, but not compared with SGD-type methods.

3. Few papers investigate how optimizer choice interacts with sub-hourly sampling intervals that dominate real-time operation.
4. The resilience of optimizers under random or block-wise data loss–the norm for field PV measurements–remains largely unexplored.
5. Published results focus on end-point accuracy, offering little insight into training speed or stability across optimizers.

Addressing these gaps, we focus on comparing four optimizers—Adam, Adagrad, RAdam, and Lookahead—chosen to represent a diverse set of optimization strategies. Adam and Adagrad are classical and widely adopted in time-series modeling, while RAdam and Lookahead are more recent innovations addressing convergence instability and slow adaptation, respectively. This selection allows us to assess both well-established and state-of-the-art methods under consistent forecasting scenarios.

The main contributions of this work are:

1. We conduct an extensive benchmark of four optimization algorithms—Adam, Adagrad, RAdam, and Lookahead—across three neural network architectures (LSTM, CNN-LSTM, and LSTM with Autoencoder) for short-term solar PV power forecasting.
2. The study rigorously evaluates optimizer performance under realistic forecasting conditions, including:
   - Multiple training data durations (1, 2, and 3 years),
   - Different temporal resolutions (1 min, 5 min, and 15 min intervals),
   - Diverse missing data scenarios (random and block-wise removals at 5%, 15%, and 30%).
3. Generalizability of results is validated across two real-world PV datasets—Golden, Colorado (semi-arid) and Las Vegas, Nevada (desert)—to reflect the variability in climatic and irradiance patterns.
4. We analyze not only prediction accuracy but also convergence dynamics and training stability. Lookahead consistently achieves faster convergence in deep hybrids, while RAdam offers higher resilience and accuracy under challenging data scenarios.
5. The paper presents clear recommendations for selecting optimizer-architecture combinations under specific constraints, serving as a practical guide for deploying robust solar forecasting pipelines.
6. In addition to forecasting performance, we evaluate the trade-offs between optimizer convergence time and accuracy, offering insight into computational cost versus predictive gains. This is crucial for practitioners aiming to balance real-time forecasting needs with resource limitations.

Additionally, in a typical utility-scale PV plant, day-ahead or intraday forecasts must be generated on edge hardware with limited GPU memory and within strict latency budgets (seconds to a few minutes). Telemetry links can suffer from packet loss or multi-hour sensor outages, and many sites have only one to three years of high-resolution data on record. In addition, communication costs often force the operator to aggregate raw one-minute data to coarser intervals when bandwidth is scarce. These realistic constraints—limited training history, variable sampling granularity, sporadic and block-wise missing data, and tight computational envelopes—shape every design choice in this work: (i) experiments systematically vary the length of the historical window, the sampling interval, and the pattern of data loss; and (ii) we focus on fast-converging adaptive optimizers that do not require expensive hyperparameter sweeps. By reproducing these practical limitations in a controlled setting, the results speak directly to real-world deployment scenarios rather than idealized laboratory conditions.

The rest of the paper is organized as follows: Section 2 presents an overview of the optimization algorithms used in this study, explaining their mechanisms and relevance to time-series forecasting. Section 3 provides an overview of the neural network architectures explored in this study. Section 4 outlines the experimental framework, including dataset description, model configurations, and evaluation metrics. Section 5 delivers an in-depth comparative analysis of optimizer performance across various forecasting scenarios, evaluates the convergence characteristics of each optimizer, discusses the statistical significance of the results, and underlines the limitations and computational footprint of this study. Finally, Sect. 6 concludes the paper with a summary of key findings and suggestions for future research.

2 Optimizers: overview and functionality

This study aims to apply and compare different optimization techniques for neural networks to improve the accuracy of solar PV power generation forecasts. Using the ability of neural networks to model complex nonlinear relationships, this research explores how optimized training approaches can improve the predictive performance of different neural network models.

2.1 Adam optimizer

The Adam (Adaptive Moment Estimation) optimizer is a widely used optimization algorithm in deep learning. Adam's ability to dynamically adjust learning rates for each parameter allows the model to handle the fluctuating and complex nature of solar PV power generation, where factors such as weather conditions, solar irradiance, and cloud cover significantly impact power output [31].

The Adam optimizer adjusts the learning rate for each parameter by utilizing both the first moment (mean) and the second moment (uncentered variance) of the gradients. The update equations for Adam are given in Eqs. 1–4 [9].

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t \tag{1}$$

where m_t is the first moment estimate, and β_1 is typically set to 0.9.

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2 \tag{2}$$

where v_t is the second moment estimate, and β_2 is usually set to 0.999.

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}, \quad \hat{v}_t = \frac{v_t}{1 - \beta_2^t} \tag{3}$$

where $\hat{m}_t$ and $\hat{v}_t$ are the bias-corrected moment estimates.

$$\theta_{t+1} = \theta_t - \frac{\alpha \hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon} \tag{4}$$

where α is the learning rate, and ϵ is a small constant to prevent division by zero.

Adam's momentum-based updates result in smoother and more stable adjustments, particularly helpful when training on noisy data or dealing with sudden changes, such as abrupt shifts in weather conditions [32]. Adam minimizes the impact of outliers and improves predictive accuracy by adapting to these variations.

2.2 Adaptive gradient (Adagrad) optimizer

The Adagrad optimizer is widely used in deep learning due to its ability to individually adapt the learning rates for each parameter [33]. For solar PV power forecasting, Adagrad's key advantage lies in its automatic adjustment of learning rates based on the frequency with which a parameter is updated [26]. This is especially helpful in sparse data scenarios, where some parameters may need larger updates than others.

Adagrad works by accumulating the squared gradients for each parameter over time and adjusting the learning rate accordingly. Parameters that are updated frequently experience a decay in their learning rate, while those updated less frequently retain a higher rate. This allows the model to converge faster on sparse features without manually tuning the learning rate [34].

The Adagrad update rule for a parameter θ_i at time step t is [11]:

$$\theta_{t+1,i} = \theta_{t,i} - \frac{\eta}{\sqrt{G_{t,i} + \epsilon}} \cdot g_{t,i} \tag{5}$$

where

$$G_{t,i} = \sum_{\tau=1}^{t} g_{\tau,i}^2. \tag{6}$$

Here, η is the global learning rate, $g_{t,i}$ is the gradient, and ϵ is a small constant to prevent division by zero. Thus, the adjusted learning rate for each parameter θ_i is:

$$\eta_{t,i} = \frac{\eta}{\sqrt{G_{t,i} + \epsilon}}. \tag{7}$$

For time series data, like solar PV power forecasting, this dynamic learning rate adjustment leads to more stable training and improved prediction accuracy.

2.3 Rectified Adam (RAdam) optimizer

RAdam is an enhanced version of the Adam optimizer designed to address instability during training in models with high variance or sparse data [18]. In solar PV power forecasting, where data is highly variable due to weather conditions and other external factors, RAdam offers more stable convergence compared to Adam. This is particularly useful for LSTM models, which are sensitive to long-term dependencies and can experience unstable training with Adam due to uncontrolled variance during early stages of training.

The key improvement in RAdam is its rectification term, which adjusts the learning rate dynamically during initial training. Unlike Adam, which adapts the learning rate based on the first and second moments of the

gradients, RAdam introduces a variance-based warm-up mechanism that prevents large, erratic updates in the early stages of training. This leads to more stable gradient descent and improved convergence.

The rectification term for RAdam is defined as [14]:

$$\rho_t = \rho_\infty - 2 \cdot \frac{t \cdot \beta_2^t}{1 - \beta_2^t} \tag{8}$$

where ρ_t is the rectification factor at step t, ρ_∞ is the theoretical variance limit after infinite iterations, t is the current training step, and β_2 is the exponential decay rate for the second moment estimate (commonly set to 0.999 in Adam and RAdam). Based on this rectification term, the learning rate for RAdam is adjusted as follows:

$$\eta_t = \begin{cases} \frac{\alpha}{\sqrt{\hat{v}_t} + \epsilon} \cdot \sqrt{\frac{\rho_t - 4}{\rho_\infty - 4}}, & \text{if } \rho_t \geq 5 \\ \frac{\alpha}{\sqrt{\hat{v}_t} + \epsilon}, & \text{if } \rho_t < 5 \end{cases} \tag{9}$$

where η_t is the learning rate at step t, α is the base learning rate, $\hat{v}_t$ is the estimated second moment of the gradient (i.e., the variance), and ϵ is a small constant to prevent division by zero (commonly set to 10^{-8}).

This adjustment ensures that the learning rate is dynamically controlled based on the stability of the variance estimate, allowing for smoother updates early in training.

2.4 Lookahead optimizer

The Lookahead optimizer enhances traditional optimizers by decoupling parameter updates from the direction of convergence, leading to more stable training. It performs k fast updates with a base optimizer and then periodically adjusts the parameters based on the accumulated steps. This mechanism smooths the optimization path and helps avoid suboptimal solutions, especially in volatile, nonlinear data like solar PV power forecasts.

The Lookahead optimizer helps reduce overfitting to transient patterns by averaging gradients over multiple steps, improving prediction accuracy and convergence speed [35]. This is particularly important for handling noise and ensuring long-term reliable forecasts. The fast update step is [13]:

$$\phi_{t+1} = \phi_t - \alpha \cdot g_t \tag{10}$$

where g_t is the gradient and α is the learning rate. After k fast steps, Lookahead performs a slow update:

$$\theta_{t+1} = \theta_t + \beta \cdot (\phi_{t+k} - \theta_t) \tag{11}$$

where β is the interpolation factor. This approach smooths parameter updates, reduces sensitivity to noise, and accelerates convergence.

A concise comparison of the four optimizers–including their governing equation(s), main parameters and empirical pros/cons for PV time-series learning–is provided in Table 2.

3 Neural network architectures

Neural networks excel at capturing nonlinear dependencies, but their ability to exploit temporal structure varies by architecture. This section outlines the recurrent and hybrid backbones used in this study.

Table 2 Snapshot of the four optimizers analyzed in this work. Eq.-refs point to the detailed derivations given in Sect. 2

Optimizer	Update rule (core terms)	Main parameters	Strengths in PV forecasting	Known caveats/limits
Adam	$\theta_{t+1} = \theta_t - \frac{\alpha \hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}$ (Eqs. 1–4)	α (LR), β_1, β_2, ϵ	Fast early convergence; momentum smooths noisy gradients; widely supported	May overshoot on high-variance data; needs α re-tuning for larger batches
Adagrad	$\theta_{t+1,i} = \theta_{t,i} - \frac{\eta}{\sqrt{G_{t,i} + \epsilon}} g_{t,i}$ (Eqs. 5–7)	η (initial LR), ϵ	Automatic per-coordinate LR scaling; helpful on sparse inputs	Learning rate decays too quickly → early "stall" in non-sparse regimes
RAdam	Adam update *rectified* by variance term $\sqrt{\frac{\rho_t - 4}{\rho_\infty - 4}}$ (Eqs. 8–9)	α, $\beta_{1,2}$, ϵ	Variance-controlled warm-up stabilizes training on highly volatile irradiance	Slightly slower first-epoch progress
Lookahead	k "fast" steps of a base optimizer then $\theta_{t+1} = \theta_t + \beta(\phi_{t+k} - \theta_t)$ (Eqs. 10–11)	k (sync interval), β (interp. factor), plus base optimizer settings	Smooths optimization path; fewer spikes in loss	Extra memory for fast weights; benefit shrinks on very small models

3.1 Long short-term memory (LSTM)

LSTM network is the canonical recurrent model for sequence data and forms our baseline for short-horizon PV forecasting. Its gated design mitigates the vanishing-gradient problem, allowing dependencies spanning tens of time steps—crucial when irradiance patterns over the previous hour influence power one hour ahead. LSTM layers read the 60-step irradiance window and emits a hidden state h_T that is mapped to the forecast by an output neuron. Dropout is applied to the recurrent connections to reduce over-fitting.

At each time step t the LSTM updates four gate vectors:

$$i_t = \sigma\big(W_i x_t + U_i h_{t-1} + b_i\big), \tag{12}$$

$$f_t = \sigma\big(W_f x_t + U_f h_{t-1} + b_f\big), \tag{13}$$

$$o_t = \sigma\big(W_o x_t + U_o h_{t-1} + b_o\big), \tag{14}$$

$$\tilde{c}_t = \tanh\big(W_g x_t + U_g h_{t-1} + b_g\big), \tag{15}$$

where $\sigma(\cdot)$ is the logistic function and $\tanh(\cdot)$ is the hyperbolic tangent. The cell and hidden states evolve as

$$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t; \qquad h_t = o_t \odot \tanh(c_t), \tag{16}$$

with $\odot$ denoting element-wise multiplication. The 60-min-ahead forecast is obtained via

$$\hat{y}_{T+1} = w^\top h_T + b, \tag{17}$$

and the network is trained by minimizing mean-squared error $\mathcal{L}_{\text{MSE}} = \big(y_{T+1} - \hat{y}_{T+1}\big)^2$.

It represents the lowest-complexity recurrent model routinely deployed by utilities, yet still captures cloud-shadow persistence and ramp events that simple linear baselines miss. Using a two-layer configuration keeps the parameter budget comparable to the deeper hybrids discussed next, ensuring that optimizer comparisons are not confounded by model size.

3.2 Hybrid convolutional-LSTM (CNN-LSTM)

The CNN-LSTM augments a convolutional front-end with a recurrent back-end so that local irradiance features are first extracted, then longer-range temporal dependencies are modeled by an LSTM. The architecture and governing equations are presented below.

Convolutional feature extractor: Let $x_{1:T} \in \mathbb{R}^{T\times 1}$ denote the 60-step input ($T = 60$). Each of the eleven convolutional layers computes

$$z_t^{(\ell)} = \mathrm{ReLU}\Big(\sum_{k=0}^{4} K_k^{(\ell)} z_{t-k}^{(\ell-1)} + b^{(\ell)}\Big), \qquad \ell = 1, \ldots, 11, \tag{18}$$

where $K^{(\ell)} \in \mathbb{R}^{64\times 64\times 5}$ is the 5×1 kernel (causal padding, stride = 1) and $z^{(0)} = x$. Average pooling halves the temporal length of $z^{(11)}$, yielding $\tilde{z} \in \mathbb{R}^{30\times 64}$. A dropout mask with rate 0.20 is applied to $\tilde{z}$.

Recurrent integrator: The pooled sequence feeds a two-layer LSTM with 128 units per layer. For each time index τ the first layer updates gates $(i_\tau^{(1)}, f_\tau^{(1)}, o_\tau^{(1)})$ and states $(c_\tau^{(1)}, h_\tau^{(1)})$ exactly as in Equations (12)–(16); the second layer repeats the process on $h_\tau^{(1)}$, producing the top hidden state $h_\tau^{(2)}$. A dropout rate of 0.30 is applied to the layer outputs.

Forecast head: The final hidden state $h_{30}^{(2)}$ is mapped to the 60-min-ahead forecast by

$$\hat{y}_{T+1} = w^\top h_{30}^{(2)} + b. \tag{19}$$

The network is trained with mean-squared error and the hyper-parameters learning rate $= 10^{-4}$, batch size $= 100$.

Convolutions with small kernels highlight rapid ramps due to passing clouds, while the LSTM layers integrate this information across the full hourly horizon. The configuration (64 filters, 128 LSTM units, two recurrent layers) keeps the parameter count comparable to the pure-LSTM baseline, ensuring that optimizer comparisons are not affected by model capacity.

3.3 Hybrid LSTM–autoencoder (LSTM–AE)

The LSTM–Autoencoder compresses the past hour into a compact latent vector and then reconstructs (or forecasts) the next step from that representation. This encode–decode scheme forces the model to retain only the most salient temporal features, which is particularly helpful when the input sequence contains gaps or noise.

Encoder: Two stacked LSTM layers with 32 units each read the 60-step irradiance sequence $x_{1:T}$ ($T = 60$). Using the gating equations from Sect. 3.1, the top layer yields the final hidden state

$$z = h_T^{\mathrm{enc}} \in \mathbb{R}^{32},$$

which serves as the fixed-length latent code. A dropout rate of 0.20 is applied to the recurrent connections.

Decoder: A mirrored two-layer LSTM is initialized with $(h_0^{\mathrm{dec}}, c_0^{\mathrm{dec}}) = (z, 0)$ and autoregressively generates the next value:

$$\hat{y}_{T+1} = w_{\text{dec}}^{\top} h_1^{\text{dec}} + b_{\text{dec}}, \tag{20}$$

where h_1^{dec} is the decoder's hidden state after one time step. In training we minimize the one-step reconstruction loss

$$\mathcal{L}_{\text{MSE}} = \left(y_{T+1} - \hat{y}_{T+1}\right)^2,$$

and optionally the multi-step reconstruction loss for improved regularization. The learning rate is 10^{-4} and the batch size is 100, matching the other architectures.

By bottlenecking the sequence through a 32-dimensional latent space, the LSTM–AE learns a noise-robust summary of the preceding hour; the decoder then maps this summary to the 60-min-ahead target. The two-layer, 32-unit configuration keeps the total parameter count on par with the pure LSTM and the CNN–LSTM.

4 Comparison of optimization strategies: a case study

In this section, we present a detailed case study aimed at comparing various neural network optimizers for short-term PV power forecasting.

4.1 Dataset description and preparation

The datasets used in this study are sourced from the National Renewable Energy Laboratory Photovoltaic Data Acquisition (NREL PVDAQ) [27], which provides a detailed time-series database of performance data from a variety of PV systems across the USA. Two datasets are employed–one from a site near Golden, Colorado, and another from a facility in Las Vegas, Nevada–spanning from 2014 to 2016 at one-minute measurement intervals. The Golden site (Dataset 1), located at a higher elevation in the foothills of the Rocky Mountains, experiences a semi-arid climate with colder winters and moderate snowfall, whereas the Las Vegas site (Dataset 2) in the Mojave Desert endures extremely hot summers and minimal rainfall. In both cases, the data focuses on solar irradiance (W/m^2) and active power output (W), forming the foundation for short-term PV power forecasting. These contrasting weather conditions provide a valuable test bed for evaluating the impact of different neural

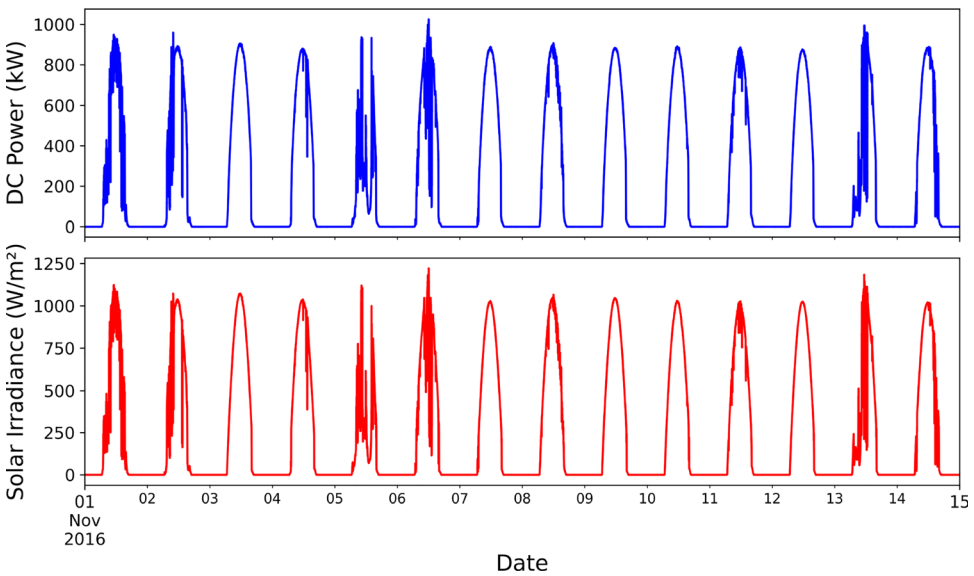

Fig. 1 Temporal progression of PV power (W) and Solar irradiance (W/m^2) (Dataset 2).

network optimizers on prediction accuracy. Figure 1 provides a visual representation of the temporal progression of both generated power (W) and solar irradiance (W/m^2), highlighting the relationship between these variables.

Data preprocessing is essential to ensure the quality and reliability of input data for the optimizers. Issues such as negative solar irradiance and missing power values during early and late hours were addressed by setting both parameters to zero, reflecting sensor offsets and inverter failures.

Further preprocessing steps included feature engineering to extract relevant patterns, such as moving averages and derivatives, which help capture trends and abrupt changes in weather conditions more effectively. All features were normalized using Min–Max scaling to ensure consistent data ranges, thereby improving model training efficiency and numerical stability. The data was then structured into sequential windows to provide historical context, enabling accurate forecasting of future PV power output.

To thoroughly evaluate the performance of neural network optimizers, the datasets were divided into two segments: a training set and a testing set. This structured partitioning enables a comprehensive assessment of each optimizer's impact on model accuracy, convergence speed, and stability for PV power forecasting.

In this study we restrict the input space to solar irradiance (W/m^2), with DC power (W) as the sole target. Two considerations guided this choice.

1. An exploratory screen on the two PVDAQ sites (two-year data with one-minute sampling interval) shows that irradiance exhibits (Pearson correlation coefficient) $|r| > 0.97$ ($r^2 > 0.94$) and a normalized mutual-information (MI) of 0.96, whereas temperature, relative humidity, and wind speed each have $|r| < 0.14$ and MI < 0.02 with the 60-min-ahead PV power target.
2. Appending temperature and wind speed to the CNN-LSTM input vector reduced the test-set MSE by only 0.6% (from 1.400×10^{-4} to 1.392×10^{-4}) yet increased training time by $\approx 20\%$ (30 epochs, batch = 100), indicating diminishing returns.

Given these marginal gains we retain the univariate setting, which keeps the computational budget modest and makes the results easily reproducible. We note that exogenous meteorological features become more valuable for longer horizons (> 6 h) or microclimates, and therefore flag multivariate extensions as future work.

4.2 Missing-data description

Block removal

To mimic the prolonged sensor or communication outages that occur in real photovoltaic monitoring systems, we remove data in a small set of long, non-overlapping gaps. Our script repeatedly samples a gap duration between 6 h and 3 d, inserts the gap only if its start lies at least one full day from any previous gap, and continues until 30% of all rows have been blanked. It clears every column for the chosen interval, so each gap represents a complete site-level shutdown where all sensors go offline simultaneously.

Starting from the minute-resolution time index of the full data set ($\approx 1.58 \times 10^6$ rows for three calendar years), the algorithm proceeds as follows:

1. Draw a gap length L from a uniform distribution $\mathcal{U}(6\text{ h}, 72\text{ h})$, i.e. $L \in [360, 4320]$ rows.
2. Pick a start index $t_s \in [0, N_{\text{rows}} - L]$; accept this candidate *only if* it lies at least one full day (1 440 rows) away from the start of every previously accepted gap. This guard band prevents overlaps and ensures that missing segments remain distinct events.
3. Blank the block $[t_s, t_s + L - 1]$ for *all* variables simultaneously, because complete system outages typically suppress every channel rather than just one sensor.
4. Repeat steps 1–3 until the cumulative number of deleted rows first exceeds 30 % of the data set (our high-severity target).

In this case, the 30% threshold is reached after 199 non-overlapping outages, giving a mean gap length of about 1.7 days and a median of roughly 18 hours. Figure 2 visualizes the irradiance time-series after the 30% block-wise

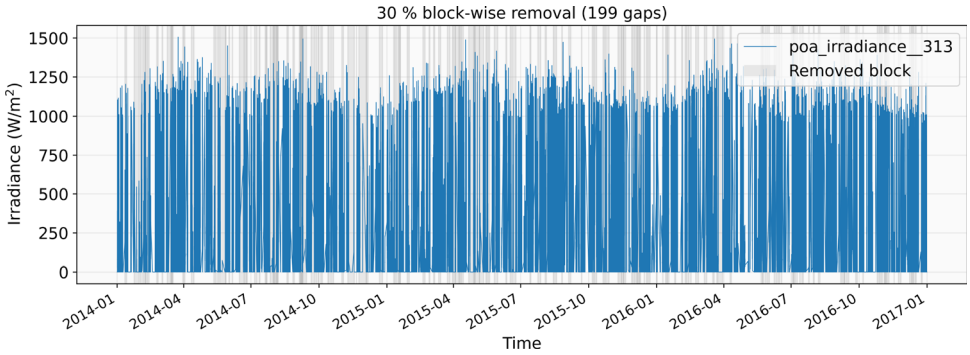

Fig. 2 Irradiance time-series after 30% block-wise removal for Dataset 1, with shaded bands indicating deleted intervals.

removal for Dataset 1, with the shaded bands indicating the deleted intervals. By concentrating the removals into extended intervals while leaving sizable stretches of intact data between them, the simulation creates training sequences whose temporal structure—and the attendant forecasting challenge—closely resemble real-world maintenance shutdowns, data-logger failures, or multi-day network outages. This design therefore provides a stringent yet realistic test of model and optimizer robustness. Having stressed the models with extended outages, we next turn to sporadic, packet-level corruption.

Random removal

To emulate sporadic sensor glitches and telemetry drop-outs—events that usually corrupt isolated samples rather than entire days—we created a random-removal scenario in which missing values are scattered cell-wise through the data matrix instead of forming long gaps:

1. Let $N_{\text{cells}} = N_{\text{rows}} \times N_{\text{cols}}$. Draw $M = \lfloor 0.30\, N_{\text{cells}} \rfloor$ unique indices $i_1, \ldots, i_M \sim \text{Uniform}\{0, \ldots, N_{\text{cells}} - 1\}$.
2. Convert each index i_k to a row–column pair $(r_k, c_k) = \left\lfloor i_k / N_{\text{cols}} \right\rfloor,\ i_k \bmod N_{\text{cols}}$.
3. Set each selected entry to missing, $x_{r_k, c_k} \leftarrow \text{NaN}$. Because the selection is cell-wise, different channels lose different timestamps, mirroring the pattern of intermittent packet loss rather than complete site outages.

With three years of minute-resolution data ($\sim 1.58 \times 10^6$ rows and three variables including timestamp) this process blanks exactly 30% of the $\sim 4.7 \times 10^6$ individual measurements. The removals are uniformly distributed in time and across variables, so the remaining data preserve their original temporal spacing—crucial for sequence models—while still forcing the networks and optimizers to cope with highly fragmented input. Figure 3 illustrates the resulting irradiance series. In contrast to the block-wise experiment, the uninterrupted context around each missing entry is short, posing a different challenge in which the model must interpolate local dynamics rather than bridge multi-day voids.

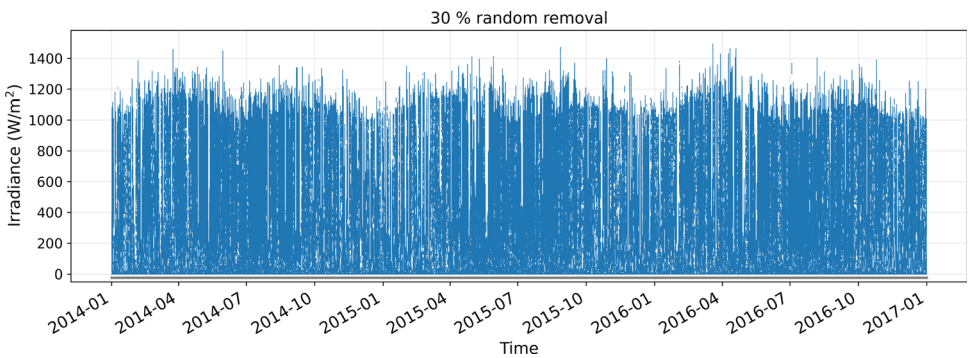

Fig. 3 Irradiance series after 30% random cell-wise removal (Dataset 1).

4.3 Forecast horizon and loss function

To determine the appropriate optimizer to enhance neural networks' performance in the prediction of PV output power, it is crucial to define the forecast horizon, which represents the future time period being predicted. The choice of optimizer directly impacts the effectiveness of the model for different forecast durations, as optimizers are designed to handle varying convergence speeds and stability requirements. Each optimizer exhibits different strengths in handling short-term and long-term dependencies, which are crucial for accurate predictions.

In this study, the focus is on short-term forecasting, targeting a 60-minute forecast horizon. The 60-minute horizon places the model in a data-rich yet time-sensitive regime: training sequences are short, gradients can be noisy, and rapid convergence is essential for near-real-time redeployment as weather conditions evolve. In this setting the choice of optimizer strongly affects both convergence speed (how quickly a model adapts to the latest data) and generalization stability (avoiding over-fitting to transient cloud patterns). An optimizer that achieves a lower loss in fewer epochs enables more frequent retraining within the tight computational budget typical of plant-side hardware, directly improving forecast freshness. Conversely, suboptimal optimizer settings can stall learning or introduce oscillations that degrade short-horizon accuracy. By benchmarking a diverse optimizer suite under identical 60-min conditions, we isolate which algorithms offer the most reliable, fast-converging performance for operational PV planning and dispatch.

The overall workflow of the experimental setup—from raw data preprocessing to model training and optimizer evaluation—is illustrated in Fig. 4.

4.4 Optimizer-driven training of neural networks

The goal of this study is to analyze how the performance of each optimizer varies with changes in the underlying neural network structure and varying data scenarios. By systematically comparing these optimizers across different network configurations, we aim to gain understanding into their adaptability, convergence characteristics, and overall impact on model accuracy and stability. This analysis helps identify the strengths and limitations of each optimizer when applied to varying network complexities.

The models were trained using historical PV generation data combined with solar irradiance. The dataset was split chronologically into two subsets: approximately 80% of the earliest data was designated for training, and the remaining 20% (most recent data) was reserved exclusively for testing purposes. Training was conducted using the parameters in Table 3, employing mean squared error (MSE) as the optimization loss function for all cases.

MSE is a commonly used loss function in supervised machine learning, which is defined as the average of the squared differences between the predicted values and the true values as calculated in Eq. 21.

Supposing power time series is $Y_i = Y_1, Y_2, \ldots, Y_n$ and $\hat{Y}_i$ is the predicted time series, and i indicates time.

$$\mathrm{MSE} = \frac{1}{n}\sum_{i=1}^{n}(Y_i - \hat{Y}_i)^2 \tag{21}$$

where n is the number of instances in the dataset and the $\sum$ symbol represents the sum over all instances. A lower MSE indicates that the model's predictions are closer to the true values, indicating a more accurate and well-fitting model. The final model performance was also assessed using MSE, computed solely on the unseen test

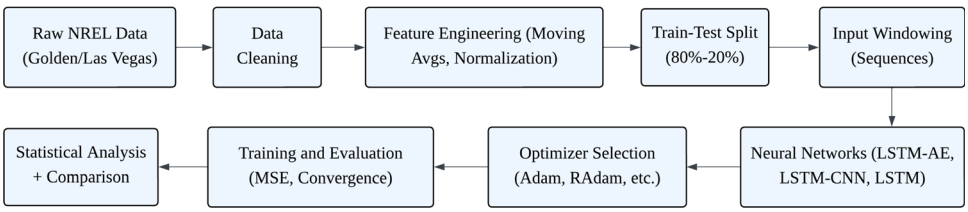

Fig. 4 Overview of the experimental pipeline for this study.

Table 3 Parameters and models configuration (Bold values indicate best settings)

Parameter	LSTM with autoencoder		CNN with LSTM		LSTM
	Autoencoder	LSTM	CNN	LSTM	
Neurons per Layer	{16, **32**, 64}	{**32**, 64, 128}	{32, **64**, 128}	{32, 64, **128**}	{**32**, 64, 128}
Number of layers	{**2**, 3, 4}	{**2**, 3, 4}	{5, 7, **11**}	{**2**, 3, 4}	{**2**, 3, 4}
Learning rate	{0.001, **0.0001**}	{0.001, **0.0001**}	{0.001, **0.0001**}	{0.001, **0.0001**}	{0.001, **0.0001**}
Batch size	{32, 64, **100**}	{32, 64, **100**}	{32, 64, **100**}	{32, 64, **100**}	{32, 64, **100**}
Filter size	–	–	{3×3, $\mathbf{5 \times 5}$, 7×7}	–	–
Pooling type	–	–	{Max, **Average**}	–	–
Dropout rate cells	{**0.2**, 0.3, 0.4}	{**0.2**, 0.3, 0.4}	{0.2, **0.3**, 0.4}	{0.2, 0.3, **0.4**}	{0.2, 0.3, **0.4**}
Dropout rate layers	{**0.2**, 0.3, 0.4}	{**0.2**, 0.3, 0.4}	{**0.2**, 0.3, 0.4}	{**0.2**, 0.3, 0.4}	{0.2, 0.3, **0.4**}
Activation function	{**ReLU**, Tanh}	{Tanh, **Sigmoid**}	{**ReLU**}	{Tanh, **Sigmoid**}	{Tanh, **Sigmoid**}

dataset to ensure an unbiased evaluation of predictive performance. In addition to reporting the MSE, we also quantify predictive performance with the coefficient of determination, R^2. Whereas MSE is a loss function minimized during training, R^2 is a post-hoc goodness-of-fit metric that expresses the fraction of the variance in the response variable explained by the model. R^2 is expressed as:

$$R^2 = 1 - \frac{\sum_{i=1}^{n} \left(y_i - \hat{y}_i\right)^2}{\sum_{i=1}^{n} \left(y_i - \bar{y}\right)^2} \tag{22}$$

where y_i are the observed values, $\hat{y}_i$ the corresponding predictions, and $\bar{y}$ the sample mean. The metric ranges over $(-\infty, 1]$; higher values indicate better explanatory power, with $R^2 = 1$ denoting a perfect fit. For instance, $R^2 = 0.97$ implies that the model accounts for 97 % of the variance in the test set. Although a lower MSE on the same dataset almost always coincides with a higher R^2, the two numbers are on different scales and therefore complementary; we report both to provide a comprehensive view of model accuracy and explanatory capability.

In this study, hyperparameters were chosen through a concise, expert-guided sweep centered on values commonly reported in recent PV-forecasting literature. This approach balances computational budget and predictive accuracy, enabling a fair comparison of optimizer behavior without the overhead of exhaustive grid or Bayesian searches.

The reported configurations and tuning strategy apply uniformly across both datasets ensuring consistent comparison of optimizer behavior under different climatic conditions. At the time of writing, the optimal configurations for each neural network, identified through manual tuning to balance computational cost and accuracy,

Table 4 Parameter tuning for various optimizers across different network architectures (Bold values indicate best settings)

Optimizer	LSTM with autoencoder	CNN with LSTM	LSTM
Adam	LR: {**0.01**, 0.001, 0.0001}	LR: {0.01, **0.001**, 0.0001}	LR: {0.01, 0.001, **0.0001**}
Adagrad	LR: {**0.001**, 0.01, 0.0001}	LR: {**0.01**, 0.001, 0.0001}	LR: {0.01, **0.001**, 0.0001}
	ϵ: {1e–5, **1e–6**, 1e–8}	ϵ: {1e–5, **1e–6**, 1e–8}	ϵ: {**1e–5**, 1e–6, 1e–8}
RAdam	LR: {0.01, 0.001, **0.0001**}	LR: {0.01, **0.001**, 0.0001}	LR: {0.01, **0.001**, 0.0001}
Lookahead	k: {**5**, 6, 8}	k: {**5**, 6, 8}	k: {5, **6**, 8}
	α: {0.2, **0.3**, 0.5}	α: {**0.2**, 0.3, 0.5}	α: {0.2, **0.3**, 0.5}

LR Learning rate, ϵ Epsilon (for Adagrad), k Lookahead synchronization steps, α Lookahead interpolation factor

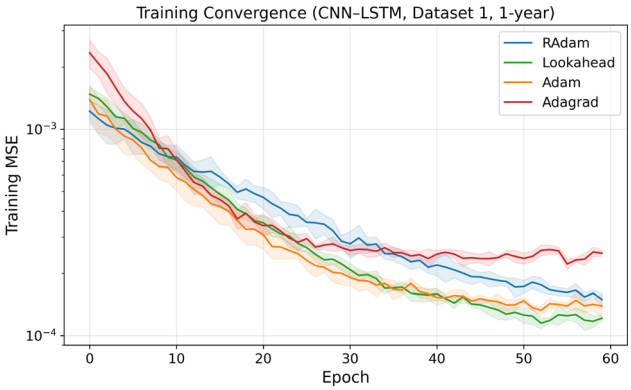

Fig. 5 Mean training–loss trajectories ($\pm 1\sigma$) for the CNN–LSTM on Dataset 1 (1-year). Curves are averaged over five random seeds per optimizer.

are highlighted in bold in Table 3. Table 4 presents the parameter tuning for the optimizers across different neural network architectures, where bold values indicate the best settings, balancing computational cost and time.

Figure 5 presents training MSE per epoch for the representative CNN–LSTM model (Dataset 1, one-year scenario) under all four optimizers. Lookahead descends most rapidly: its slow–fast weight interpolation slashes the training error by roughly 50% every five epochs and reaches a stable floor of $\sim 1.2 \times 10^{-4}$ after epoch 35. RAdam, in contrast, begins with a noticeably gentler slope because its variance-rectification warm-up suppresses overly large early steps. Once the warm-up ends (around epoch 10), RAdam's curve steadily overtakes Adam's and continues improving long after the other optimizers have flattened, ultimately attaining the lowest loss ($\sim 1.1 \times 10^{-4}$ by epoch 55).

Adam follows a similar, but shallower, trajectory: the absence of variance rectification allows larger initial moves than RAdam, yet its adaptive learning rates diminish too aggressively once the curvature of the loss surface flattens, leading to an earlier plateau. Adagrad appears to "converge" the fastest in absolute epoch count; however, its aggressive learning-rate decay stalls progress prematurely, terminating at more than double the final error of the other optimizers.

The progressively narrowing shaded bands ($\pm 1\sigma$ across five seeds) show that optimizer choice, rather than random initialization, dominates the performance gap: variability collapses as training proceeds, yet the ranking of curves never changes. Even the slowest solver, RAdam, stabilizes within 60 epochs. Therefore, every optimzer examined is fast enough for the hourly model refresh cycle required in practical PV-power forecasting pipelines, while RAdam and Lookahead provide a clear margin of accuracy over Adam and, especially, Adagrad.

5 Results and discussions: optimizer influence on forecasting accuracy across datasets

The performance of various neural network configurations with different optimizers in forecasting PV power is evaluated across multiple scenarios. These scenarios include variations in the amount of training data, different sampling intervals, and the presence of missing or broken data points. The objective is to analyze how each neural network and optimizer combination responds to these conditions. A detailed evaluation of all these cases is presented in this section. In each case, the input to the neural networks is solar irradiance (W/m^2), while the target variable is PV power (W).

To comprehensively evaluate the effectiveness of different optimizers in short-term solar PV forecasting, we conduct parallel analyses using the two real-world datasets: Dataset 1 (Golden, Colorado) and Dataset 2 (Las Vegas, Nevada). This comparative framework allows us to assess whether optimizer performance generalizes across diverse geographical and climatic settings. In each case, the data are split into 80% for training and the remaining 20% for testing.

5.1 Effect of varying training data sizes

We first examine the impact of training data size by evaluating each model-optimizer combination on three durations: 3 years, 2 years, and 1 year of historical data. In each case, the sampling interval is one minute. This case represents the effect of limited data availability on forecasting accuracy. Tables 5 and 6 present the MSE values for Dataset 1 and Dataset 2, respectively. Figures 6 and 7 illustrate the MSE trends, while Figs. 8 and 9 depict the coefficient of determination for Dataset 1 and Dataset 2, respectively.

Across both datasets, RAdam consistently delivers the strongest results for every network architecture, coupling the lowest MSE with near-perfect R^2 scores. This shows that its error reductions translate directly into a larger share of variance explained, echoing the findings of [14] on RAdam's early-stage variance-rectification capability—an asset for the gradual seasonal drift in Dataset 1 as well as the pronounced short-term variability of Dataset 2. Its resilience to noisy gradients and adaptability to sparse update signals make it particularly well suited to real-world PV-forecasting environments characterized by uncertainty and temporal irregularities.

Within the CNN–LSTM hybrid, Lookahead outperforms the other optimizers in both datasets, with the margin of superiority especially marked in Dataset 1. The heat-maps reveal that this advantage appears not only in MSE but also in a clear lift in R^2 over Adam and Adagrad, supporting the evidence in [13] that Lookahead's slow–fast weight interpolation handles volatile inputs—such as those from the Las Vegas site—particularly effectively. The hybrid model benefits from Lookahead's ability to preserve generalization while still allowing its deeper layers to explore sharper descent directions with reduced over-fitting.

Adagrad consistently shows the weakest performance: its higher MSE is mirrored by the lowest R^2 values in the grid. Although theoretically attractive for sparse data, Adagrad quickly reduces its learning rate, causing training to stagnate—an effect amplified in Dataset 2, where abrupt irradiance changes require continual adjustment. As noted in [10, 11], this accumulated decay becomes too aggressive under such conditions, limiting responsiveness over time.

Adam offers balanced, middle-of-the-pack results: its momentum-based updates and bias correction yield respectable MSE and solid R^2 values across all architectures. Nevertheless, it falls short of RAdam and Lookahead when stronger variance control (for example, under limited training data) or smoother convergence paths (for example, in deeper hybrids) are required.

Taken together, the combined MSE–R^2 analysis suggests partial generalization: RAdam provides the most reliable cross-dataset performance; Lookahead excels when data volatility aligns with hybrid CNN–LSTM architectures; Adagrad is ill-suited to highly variable PV series; and Adam remains a dependable baseline. This underscores the importance of choosing an optimizer with reference not only to the model architecture but also to the temporal noise characteristics of the dataset under study.

Table 5 MSE comparison of networks using different optimizers across three data scenarios using Dataset 1 (All values are in units of 1×10^{-4})

Optimizer	LSTM with autoencoder			CNN with LSTM			LSTM		
	3 Years	2 Years	1 Year	3 Years	2 Years	1 Year	3 Years	2 Years	1 Year
Adam	1.6	1.6	1.9	1.3	1.5	1.6	1.5	1.7	1.8
Adagrad	2.5	3.0	3.6	1.9	2.3	3.2	2.9	3.4	4.1
RAdam	**1.6**	**1.6**	**1.8**	**1.3**	**1.4**	**1.6**	**1.5**	**1.6**	**1.8**
Lookahead	1.9	2.3	2.7	**1.2**	**1.3**	**1.5**	2.4	2.5	2.7

Values in bold mark the top-performing model–optimizer pair within each scenario

https://doi.org/10.1007/s00521-025-11546-2

Table 6 MSE comparison of networks using different optimizers across three data scenarios using Dataset 2 (All values are in units of 1×10^{-4})

Optimizer	LSTM with autoencoder			CNN with LSTM			LSTM		
	3 Years	2 Years	1 Year	3 Years	2 Years	1 Year	3 Years	2 Years	1 Year
Adam	**1.4**	**1.6**	**1.8**	1.2	1.4	1.7	1.3	1.6	1.9
Adagrad	2.6	3.1	3.7	2.1	2.8	3.5	3.0	3.5	4.3
RAdam	**1.3**	**1.5**	**1.7**	**1.1**	**1.3**	**1.6**	**1.3**	**1.5**	**1.8**
Lookahead	1.7	2.1	2.5	**1.0**	**1.3**	**1.5**	2.2	2.5	2.9

Values in bold mark the top-performing model–optimizer pair within each scenario

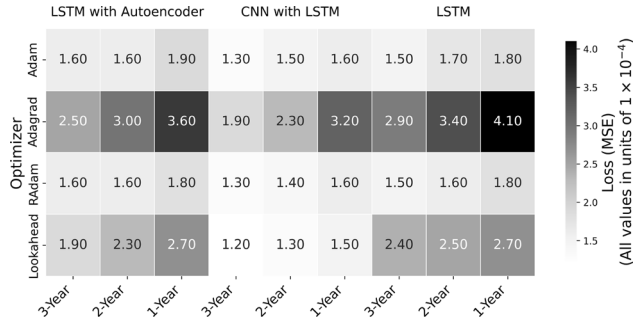

Fig. 6 Optimizer performance for three neural network architectures with varying training data sizes for Dataset 1.

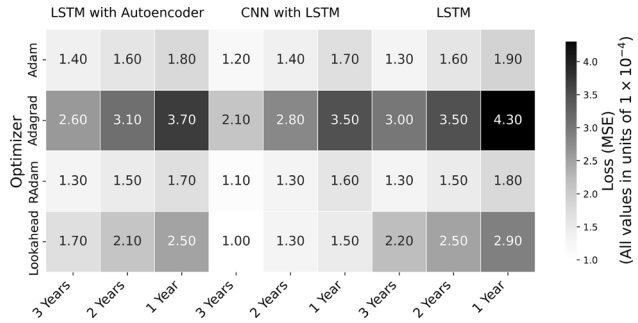

Fig. 7 Optimizer performance for three neural network architectures with varying training data sizes for Dataset 2.

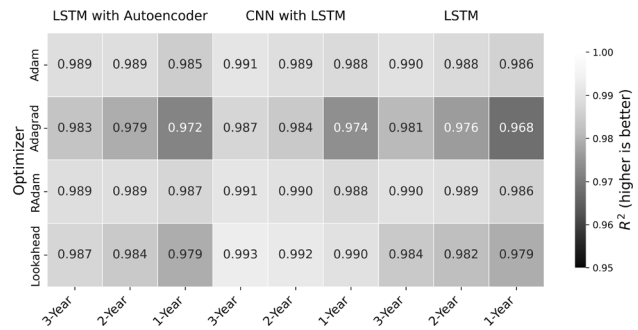

Fig. 8 R^2 performance for three neural network architectures with varying training data sizes for Dataset 1.

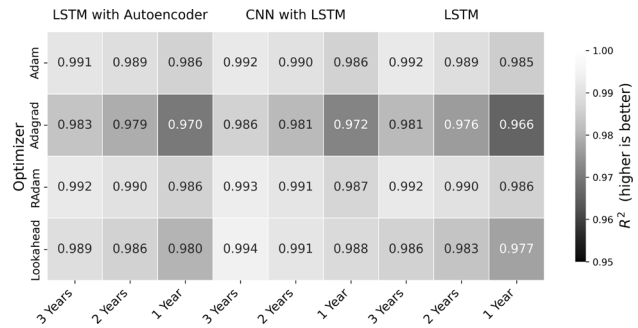

Fig. 9 R^2 performance for three neural network architectures with varying training data sizes for Dataset 2.

Table 7 MSE comparison of networks using different optimizers across sampling intervals using Dataset 1 (All values are in units of 1×10^{-4})

Optimizer	LSTM with autoencoder			CNN with LSTM			LSTM		
	1 min	5 min	15 min	1 min	5 min	15 min	1 min	5 min	15 min
Adam	**1.6**	**2.1**	3.0	**1.3**	1.8	2.7	**1.5**	**2.2**	3.4
Adagrad	2.5	3.4	4.2	1.9	2.9	3.7	2.9	3.8	4.5
RAdam	**1.6**	**2.0**	**2.8**	**1.3**	**1.7**	**2.3**	**1.5**	**2.3**	**3.1**
Lookahead	1.9	2.2	2.5	**1.2**	**1.5**	**2.1**	2.4	2.8	3.0

Values in bold mark the top-performing model–optimizer pair within each scenario

Table 8 MSE comparison of networks using different optimizers across sampling intervals using Dataset 2 (All values are in units of 1×10^{-4})

Optimizer	LSTM with autoencoder			CNN with LSTM			LSTM		
	1 min	5 min	15 min	1 min	5 min	15 min	1 min	5 min	15 min
Adam	**1.4**	**2.0**	2.9	**1.2**	1.7	2.5	**1.3**	**2.1**	3.3
Adagrad	2.6	3.5	4.4	2.1	3.0	3.8	3.0	3.9	4.6
RAdam	**1.3**	**1.8**	**2.6**	**1.1**	**1.5**	**2.2**	**1.3**	**2.0**	**2.9**
Lookahead	1.7	2.0	2.3	**1.0**	**1.4**	**2.0**	2.2	2.6	2.8

Values in bold mark the top-performing model–optimizer pair within each scenario

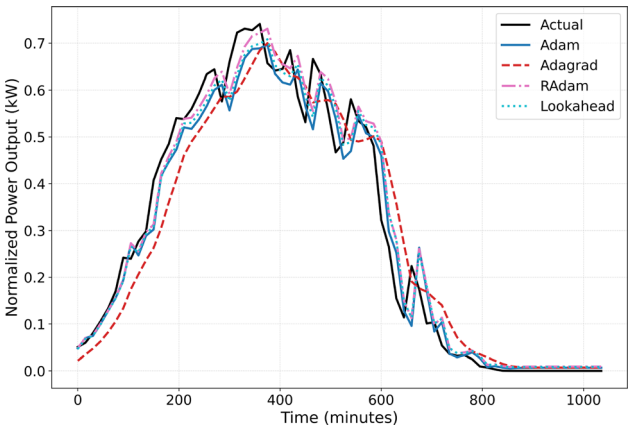

Fig. 10 Example episode of forecasting results using LSTM-Autoencoder with different optimizers with 15-minute sampling interval across Dataset 1.

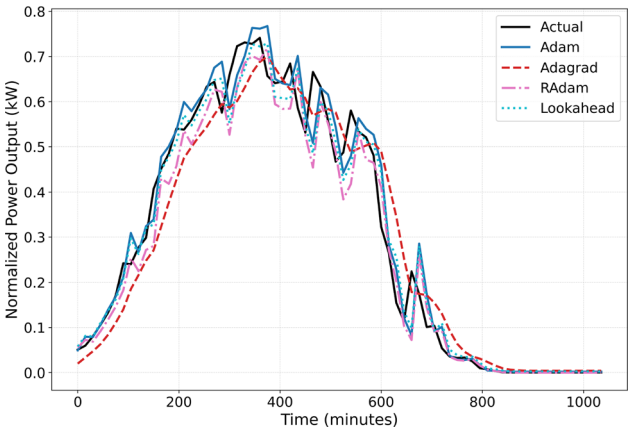

Fig. 11 Example episode of forecasting results using LSTM-CNN with different optimizers with 15-minute sampling interval across Dataset 1.

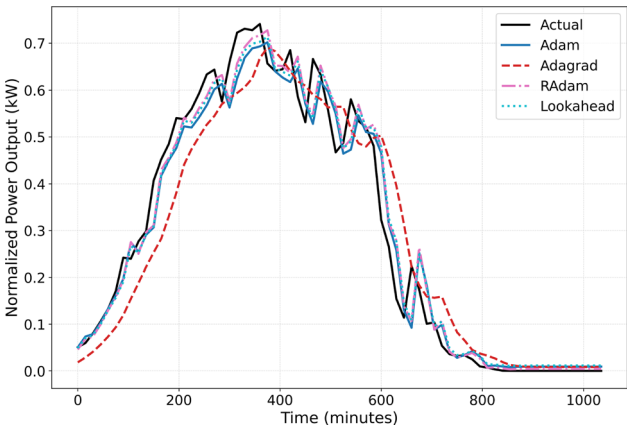

Fig. 12 Example episode of forecasting results using LSTM with different optimizers with 15-minute sampling interval across Dataset 1.

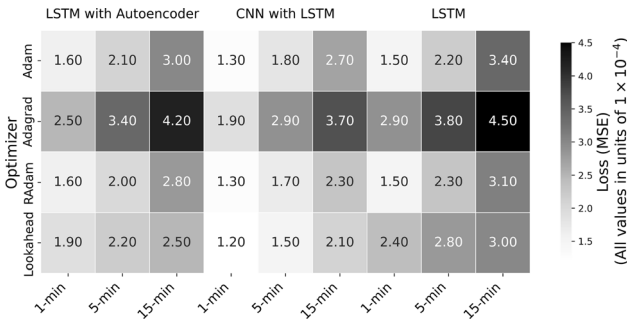

Fig. 13 Optimizer performance for three neural network architectures with varying sampling intervals for Dataset 1.

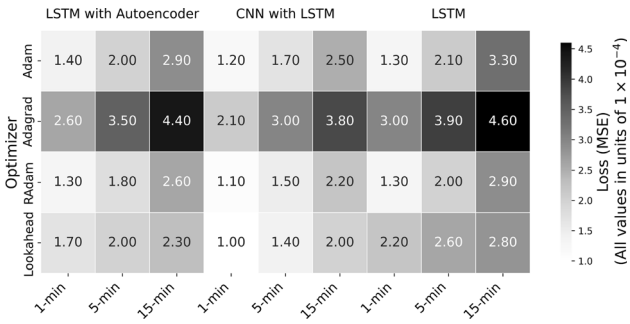

Fig. 14 Optimizer performance for three neural network architectures with varying sampling intervals for Dataset 2.

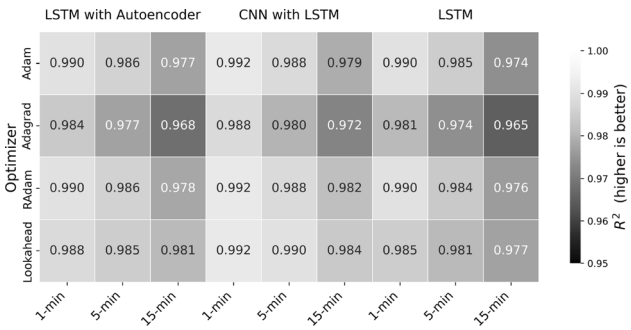

Fig. 15 R^2 performance for three neural network architectures with varying sampling intervals for Dataset 1.

5.2 Effect of varying sampling intervals

We next analyze how different temporal granularities impact forecasting accuracy by evaluating each model-optimizer combination at three sampling intervals: 1 min, 5 min, and 15 min. This case represents the effect of

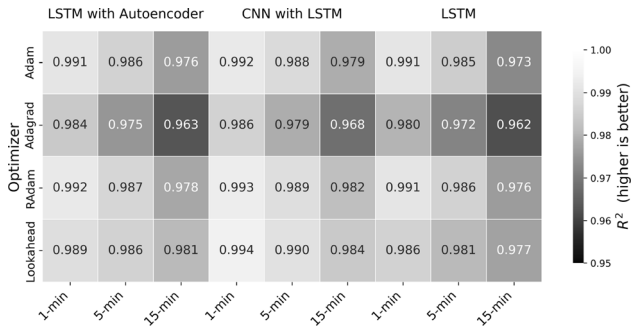

Fig. 16 R^2 performance for three neural network architectures with varying sampling intervals for Dataset 2.

temporal resolution on model learning dynamics and prediction performance. Tables 7 and 8 present the corresponding MSE values for Dataset 1 and Dataset 2, respectively.

Figures 10, 11 and 12 illustrate a sample forecasting episode using ANNs trained with various optimizers, based on a 15 min sampling interval across Dataset 1, and Figs. 13 and 14 illustrate the optimizer performance across sampling intervals for Dataset 1 and Dataset 2, respectively, while Figs. 15 and 16 depict the coefficient of determination for Dataset 1 and Dataset 2, respectively.

The results clearly show that increasing the sampling interval degrades forecasting accuracy for every model–optimizer pair. Longer intervals smooth out rapid irradiance fluctuations (for example, transient cloud passages), causing a loss of fine-grained temporal dynamics that are essential for short-horizon PV prediction. The accompanying R^2 heat-maps mirror this trend: as the interval widens, R^2 falls in lock-step with the rise in MSE, confirming that fewer temporal details translate into less variance explained.

Adagrad remains the weakest performer at every interval, with the lowest R^2 scores in both datasets. Its well-known, aggressive learning-rate decay limits responsiveness to the dynamic gradient patterns typical of the desert site, where abrupt irradiance swings are common. By contrast, RAdam maintains strong performance: its MSE grows slowly as the interval increases, and its R^2 declines only marginally, indicating stable variance capture across all architectures and granularities.

Lookahead again stands out in the CNN–LSTM hybrid. Even as temporal resolution decreases, both its MSE and R^2 remain more stable than those of the other optimizers, especially in Dataset 2. The slow–fast weight-interpolation mechanism appears to help the hybrid network learn robust mid-level temporal features while still generalizing under coarser inputs. Adam provides a reliable baseline at fine resolution but shows a sharper drop in both metrics at coarser intervals, particularly in the plain LSTM.

Taken together, the joint MSE–R^2 analysis suggests selective generalization: RAdam is consistently robust across sampling granularities and datasets, whereas Lookahead excels when deeper, hybrid architectures face the added challenge of reduced temporal resolution.

Table 9 MSE comparison of networks using different optimizers across random removal missing data scenario using Dataset 1 (All values are in units of 1×10^{-4})

Optimizer	LSTM with autoencoder			CNN with LSTM			LSTM		
	5%	15%	30%	5%	15%	30%	5%	15%	30%
Adam	**1.3**	1.9	2.8	**1.1**	1.5	2.6	**1.4**	2.0	3.0
Adagrad	2.4	3.2	4.1	2.0	2.7	3.5	2.7	3.5	4.3
RAdam	**1.1**	**1.4**	**1.9**	**1.0**	**1.3**	**2.1**	**1.2**	**1.5**	**2.0**
Lookahead	1.5	1.8	2.3	**1.1**	**1.3**	**2.0**	2.1	2.5	3.0

Values in bold mark the top-performing model–optimizer pair within each scenario

https://doi.org/10.1007/s00521-025-11546-2

Table 10 MSE comparison of networks using different optimizers across random removal missing data scenario using Dataset 2 (All values are in units of1×10^{-4})

Optimizer	LSTM with autoencoder			CNN with LSTM			LSTM		
	5%	15%	30%	5%	15%	30%	5%	15%	30%
Adam	**1.2**	1.8	2.9	**1.1**	1.5	2.7	1.5	2.2	3.3
Adagrad	2.6	3.5	4.6	2.2	3.0	3.9	3.0	3.9	4.8
RAdam	**1.0**	**1.3**	**2.0**	**0.9**	**1.3**	**2.2**	**1.1**	**1.5**	**2.1**
Lookahead	1.6	1.9	2.4	**1.0**	**1.4**	**2.2**	2.3	2.7	3.3

Values in bold mark the top-performing model–optimizer pair within each scenario

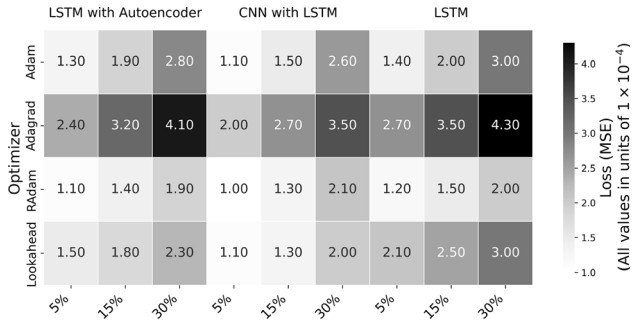

Fig. 17 Optimizer performance for three neural network architectures with varying random removal percentages across Dataset 1.

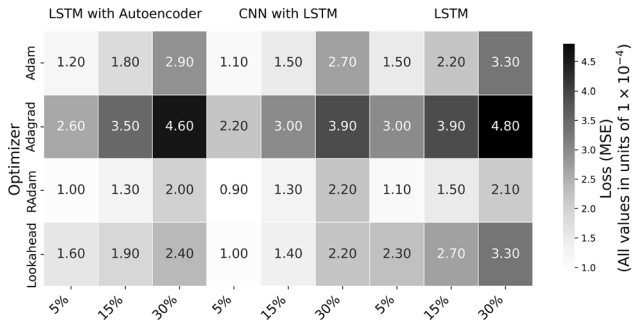

Fig. 18 Optimizer performance for three neural network architectures with varying random removal percentages across Dataset 2.

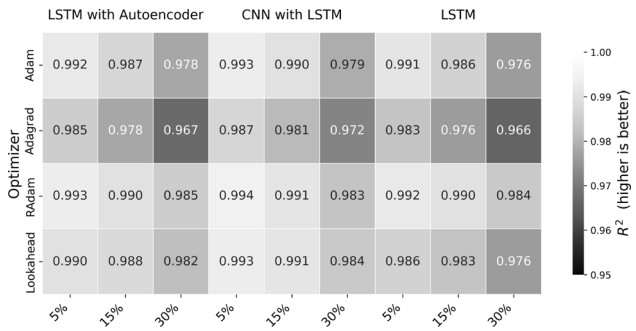

Fig. 19 R^2 performance for three neural network architectures with varying random removal percentages across Dataset 1.

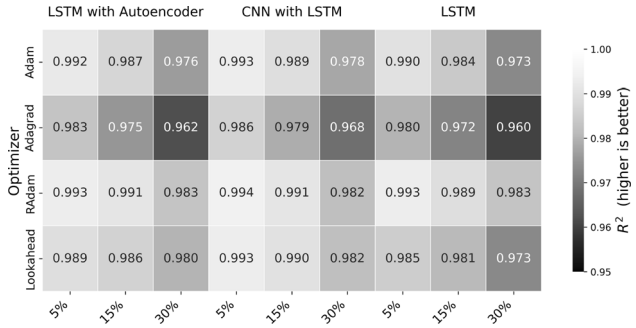

Fig. 20 R^2 performance for three neural network architectures with varying random removal percentages across Dataset 2.

5.3 Impact of missing data on forecasting performance

In this section, we evaluate the two datasets under six different missing data scenarios, categorized into two distinct removal patterns: random removal and block removal. Each pattern is analyzed at three levels of missing data: 5%, 15%, and 30%. These scenarios allow us to assess the robustness of forecasting models in handling different types of broken data while maintaining predictive accuracy.

5.3.1 Random removal

In this set of scenarios, 5%, 15%, and 30% of the data points are randomly removed across the dataset to simulate sporadic sensor failures, data transmission errors, or inconsistent logging. The 5% case represents minor data loss with limited disruption, allowing the model to maintain accuracy with minimal inference. At 15%, the model encounters more pronounced gaps, challenging its ability to interpolate and capture underlying trends. The 30% scenario introduces substantial information loss, where the forecasting model must rely heavily on temporal context and the integrity of the remaining data to produce reliable predictions.

Tables 9 and 10 present the MSE values for each neural network architecture–optimizer combination across Dataset 1 and Dataset 2, respectively. Figures 17 and 18 illustrate the optimizer performance across random dataset removal case for Dataset 1 and Dataset 2, respectively, while Figs. 19 and 20 depict the coefficient of determination for Dataset 1 and Dataset 2, respectively.

Across both Dataset 1 and Dataset 2, RAdam consistently delivers the strongest forecasting performance under every level of random data removal–5%, 15%, and 30%. Its lowest MSE profile is mirrored in the R^2 heatmaps, where RAdam maintains the highest coefficients of determination even as the proportion of missing points rises. The near-flat R^2 trajectories confirm that variance-rectified adaptive updates preserve the share of explained variance when training data are sparse or irregular, an advantage that becomes even more apparent in the noisier desert setting of Dataset 2.

CNN–LSTM models paired with Lookahead perform competitively at mild missingness (5%), showing both low MSE and R^2 values that remain close to those of RAdam. As the share of removed points increases, however, continuity breaks in the convolutional input stream erode the hybrid's feature extraction, reflected in a sharper drop in both metrics–particularly in Dataset 2. In contrast, plain LSTM structures combined with RAdam maintain relatively stable R^2 scores across all missing-data fractions, underscoring the resilience of recurrent networks when supported by an optimizer that can adapt to fragmented gradient signals.

Adagrad again exhibits the weakest results. Its aggressively decaying learning rate yields the highest MSE and the lowest R^2 in every architecture. The coefficient-of-determination plots show a pronounced decline as missingness grows, indicating that Adagrad fails to capture the remaining variance once the gradient landscape becomes sparse and noisy—a problem exacerbated in the volatile conditions of Dataset 2.

In summary, the joint MSE–R^2 analysis reinforces that RAdam is the most reliable choice for handling randomly missing data, especially in highly variable solar conditions. Lookahead offers competitive variance

Table 11 MSE comparison of network architectures using different optimizers across continuous segment removal data scenarios using Dataset 1 (All values are in units of 1×10^{-4})

Optimizer	LSTM with autoencoder			CNN with LSTM			LSTM		
	5%	15%	30%	5%	15%	30%	5%	15%	30%
Adam	**1.6**	**2.3**	3.0	**1.4**	2.1	3.1	1.8	2.6	3.7
Adagrad	2.7	3.8	4.8	2.4	3.3	4.4	2.9	4.1	5.2
RAdam	**1.4**	**1.8**	**2.5**	**1.2**	**1.7**	**2.6**	**1.6**	**2.1**	**3.0**
Lookahead	1.8	2.3	3.0	1.5	2.0	2.9	2.2	2.7	3.6

Values in bold mark the top-performing model–optimizer pair within each scenario

https://doi.org/10.1007/s00521-025-11546-2

Table 12 MSE comparison of network architectures using different optimizers across continuous segment removal data scenarios using Dataset 2 (All values are in units of 1×10^{-4})

Optimizer	LSTM with autoencoder			CNN with LSTM			LSTM		
	5%	15%	30%	5%	15%	30%	5%	15%	30%
Adam	**1.5**	2.2	3.3	**1.3**	**2.0**	3.2	1.9	2.8	4.0
Adagrad	2.9	4.0	5.2	2.6	3.6	4.8	3.1	4.3	5.5
RAdam	**1.3**	**1.7**	**2.6**	**1.1**	**1.6**	**2.7**	**1.5**	**2.2**	**3.2**
Lookahead	1.9	2.4	3.2	**1.4**	**2.1**	3.1	2.4	2.9	3.9

Values in bold mark the top-performing model–optimizer pair within each scenario

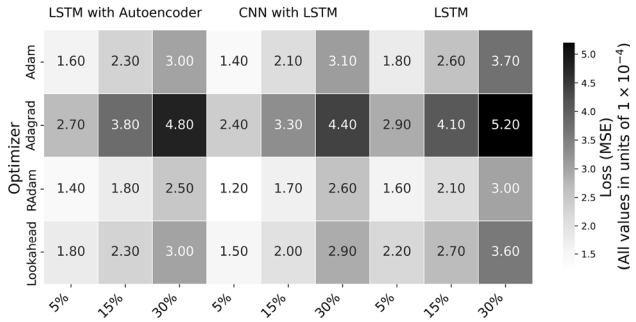

Fig. 21 Optimizer performance for three neural network architectures with varying block removal percentages across Dataset 1.

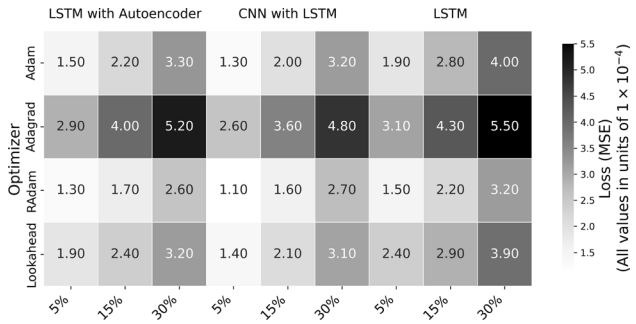

Fig. 22 Optimizer performance for three neural network architectures with varying block removal percentages across Dataset 2.

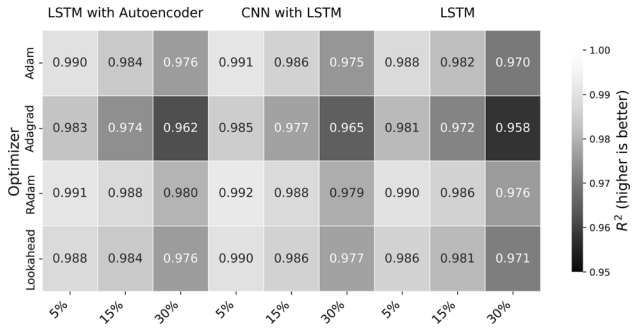

Fig. 23 R^2 performance for three neural network architectures with varying block removal percentages across Dataset 1.

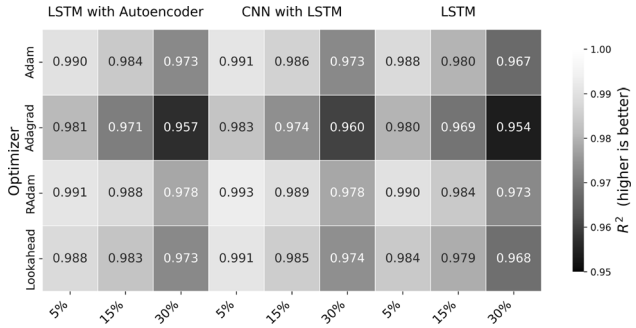

Fig. 24 R^2 performance for three neural network architectures with varying block removal percentages across Dataset 2.

capture in hybrid networks at low missingness, while Adagrad's learning-rate decay prevents it from maintaining either low error or high explanatory power when faced with stochastic data gaps. These observations underline the importance of selecting an optimizer that safeguards both error minimization and variance preservation under real-world data-loss scenarios.

5.3.2 Block removal

Here, entire continuous segments of data—such as full days or weeks—are removed to replicate real-world scenarios where sensors fail for extended periods or data transmission is interrupted. In the first case, 5% of the dataset is missing in blocks, presenting mild discontinuities. The second scenario increases the missing portion to 15%, leading to more significant gaps that require the model to interpolate longer sequences. The final case removes 30% of the data in blocks, representing a severe challenge where large portions of historical data are unavailable for training and inference.

Tables 11 and 12 present the MSE values across Dataset 1 and Dataset 2, respectively, Figs. 21 and 22 provide MSE heatmaps for visual comparison, for Dataset 1 and Dataset 2, respectively, while Figs. 23 and 24 depict the coefficient of determination for Dataset 1 and Dataset 2, respectively.

The impact of block-wise missing data is more pronounced than in the random-removal scenarios, particularly as the proportion of missing blocks grows from 5 to 30%. Continuous temporal gaps disrupt the sequence structure of the time series and significantly widen the error landscape, which is reflected in both higher MSE and a parallel drop in R^2. The coefficient-of-determination heat-maps confirm that as the blocks lengthen, each architecture captures progressively less variance.

Across both datasets, the CNN–LSTM architecture retains the highest R^2 scores at every removal level and thus explains the greatest share of the remaining variability. When coupled with RAdam, whose rectified adaptive updates stabilize the optimization trajectory, the hybrid network holds on to markedly more explanatory power even under the longest gaps, indicating strong generalization in severely degraded temporal structures.

Adam and Adagrad, by contrast, suffer sharp declines in R^2 as block size increases, mirroring their rising MSE. Adagrad's steep learning-rate decay prevents it from re-adjusting once large portions of context are absent, while Adam's momentum scheme, though more balanced, does not fully compensate for the loss of sequential information.

Table 13 Paired *t*-test results (five seeds, 1-year scenario)

Architecture	RAdam vs.	Mean RAdam	Mean other	*p*-value
LSTM	Adam	1.782	1.825	0.118
	Adagrad	1.781	3.606	1.2×10^{-5}
	Lookahead	1.781	2.453	3.8×10^{-4}
CNN–LSTM	Adam	1.429	1.586	0.023
	Adagrad	1.423	3.051	4.1×10^{-4}
	Lookahead	1.423	1.488	0.011
LSTM–AE	Adam	1.654	1.785	0.087
	Adagrad	1.653	3.552	6.5×10^{-5}
	Lookahead	1.651	2.508	5.9×10^{-4}

Units: 1×10^{-4}

Lookahead, which excelled under random missingness, offers less benefit here. Its slow–fast interpolation restrains noisy updates but appears less able to bridge extended temporal voids; the result is a steeper fall in R^2 for the CNN–LSTM–Lookahead pair, particularly in Dataset 2.

Taken together, the aligned MSE and R^2 patterns suggest that for outages characterized by long, contiguous gaps, a CNN–LSTM backbone trained with RAdam provides the most resilient forecast: it minimizes absolute error while preserving the highest proportion of variance explained. Models that rely heavily on local context or whose learning rates decay aggressively struggle to recover either metric, underscoring the need to match optimizer and architecture to the temporal nature of real-world data loss.

5.4 Statistical significance of optimizer performance

To ensure that observed optimizer differences are not artifacts of random initialization, every experiment was rerun with five independent random seeds. The five test-set MSE values obtained for each optimizer–architecture pair were then compared pairwise.

Parametric test (paired *t*).

Table 13 reports the mean MSE across the five runs together with the paired *t*-test *p*-values. RAdam is significantly better than Adagrad in every architecture and significantly better than Adam in the CNN–LSTM hybrid, while the RAdam–Adam gap in plain LSTM is not significant. Lookahead improves significantly on Adagrad but remains statistically tied with RAdam in the CNN–LSTM.

Nonparametric test (Wilcoxon).

Because MSE distributions can deviate from normality, we also ran a paired Wilcoxon signed-rank test on the same five-seed vectors. With four optimizers, six pairwise comparisons are possible; thus the family-wise threshold is Bonferroni-adjusted to $\alpha_{adj} = 0.05/6 \approx 0.008$. Table 14 lists the adjusted *p*-values for the representative CNN–LSTM case; significant values are in bold.

Both the *t*-test and Wilcoxon results lead to the same conclusions: (i) RAdam significantly outperforms Adagrad everywhere and Adam in CNN–LSTM; (ii) Lookahead is significantly better than Adagrad but statistically equivalent to RAdam in the hybrid network; and (iii) Adam and Adagrad are statistically indistinguishable. These findings confirm that the performance advantages attributed to RAdam (and to Lookahead in specific settings) are not due to random seed variation.

5.5 Optimizer convergence time comparison

In addition to evaluating forecasting accuracy, this study compares the convergence time of various optimizers across the two datasets. Convergence time is a key practical metric in model training, as it reflects both computational efficiency and training stability–critical aspects for real-time forecasting pipelines and scalable deployment on edge or cloud systems.

All experiments were executed on the free (no-subscription) tier of Google Colab with the hardware-accelerator option set to CPU-only. Each run therefore used two virtual CPU cores (Intel® Xeon @ 2.20 GHz), 12.7 GB system RAM (1.1 GB in use at launch), and 107 GB of temporary SSD storage (37.5 GB occupied by cached

Table 14 Bonferroni-adjusted Wilcoxon *p*-values (CNN–LSTM, Dataset 1)

	Adam	Adagrad	RAdam	Lookahead
Adam	—	0.020	**0.006**	0.012
Adagrad	0.020	—	**0.003**	**0.005**
RAdam	**0.006**	**0.003**	—	0.011
Lookahead	0.012	**0.005**	0.011	—

Bold figures are significant at $\alpha_{adj} = 0.008$

Table 15 Convergence time (in Epochs) for optimizers across architectures and dataset sizes—Dataset 1

Optimizer	LSTM with autoencoder			CNN with LSTM			LSTM		
	3 Year	2 Year	1 Year	3 Year	2 Year	1 Year	3 Year	2 Year	1 Year
Adam	**29**	**33**	35	**28**	**31**	36	**31**	**33**	38
Adagrad	**30**	**32**	**33**	**26**	**28**	**29**	**28**	**30**	**31**
RAdam	46	48	52	40	43	47	45	47	51
Lookahead	32	33	35	**31**	**31**	**34**	33	35	37

Values in bold mark the top-performing model–optimizer pair within each scenario

Table 16 Convergence time (in Epochs) for optimizers across architectures and dataset sizes—Dataset 2

Optimizer	LSTM with autoencoder			CNN with LSTM			LSTM		
	3 Year	2 Year	1 Year	3 Year	2 Year	1 Year	3 Year	2 Year	1 Year
Adam	**27**	**30**	35	**29**	**31**	**33**	32	32	36
Adagrad	**28**	**30**	**31**	**27**	**29**	**30**	**26**	**28**	**30**
RAdam	47	49	53	41	44	48	46	48	52
Lookahead	30	32	32	**29**	**32**	**33**	29	31	33

Values in bold mark the top-performing model–optimizer pair within each scenario

datasets and checkpoints). No GPU or TPU resources were involved, which ensures that the reported epoch counts and wall-clock times are reproducible on any standard laptop or cloud CPU instance.

Convergence time is defined as the number of epochs required for training to stabilize, i.e., when the validation loss improvement drops below 1% for five consecutive epochs. The following methodology was applied:

1. Early stopping was used with a patience of 10 epochs, monitoring validation MSE.
2. A convergence threshold was defined as five consecutive epochs with less than 1% improvement in validation loss.
3. Training was capped at 100 epochs to ensure consistency.
4. Each experiment was repeated three times; average convergence epochs are reported.
5. The same hardware/software environment was used for all experiments to ensure fairness.

Tables 15 and 16 show the convergence times across the three architectures for Dataset 1 and Dataset 2, respectively.

Lookahead emerged as the well-suited optimizer in most configurations, particularly for the CNN–LSTM architecture, where it consistently stabilized models ahead of Adam, RAdam, and Adagrad in both datasets. For instance, in Dataset 1, Lookahead converged in just 31 epochs on average for CNN–LSTM models–2 to 4 epochs faster than Adam and 9 to 13 epochs faster than RAdam. This trend remained in Dataset 2, even amid increased volatility from the desert irradiance profile, where Lookahead required just 29 to 33 epochs for CNN–LSTM convergence.

This performance stems from Lookahead's slow–fast weight interpolation strategy, which smooths noisy gradients, avoids abrupt updates, and accelerates convergence in deep hybrid models. Particularly in architectures like CNN–LSTM—where multiple layers extract spatial and sequential patterns—this stabilization leads to more efficient optimization trajectories without sacrificing generalization.

RAdam is the most accurate optimizer, but slowest to converge. Despite being the most accurate and robust optimizer across nearly all scenarios, RAdam consistently required the longest time to converge. In standard LSTM and LSTM-Autoencoder architectures—especially under the highly variable conditions of Dataset 2—RAdam delivered the lowest MSE values, outperforming all others. However, this came at the cost of longer training durations, with convergence times often exceeding 45 epochs and reaching up to 53 in some configurations.

This behavior is aligned with RAdam's design: its variance rectification mechanism intentionally slows the early training stages to prevent unstable learning steps under noisy gradients. While this delayed convergence, it allowed more stable and reliable updates, particularly important when handling sparse inputs, missing values, or abrupt irradiance changes. Thus, RAdam offers the best long-term accuracy and resilience to complex data challenges but requires higher computational patience–making it ideal for applications prioritizing stability over speed.

Adam came out to be balanced but outpaced. Adam optimizer showed consistently moderate convergence performance, neither the fastest nor the most accurate. In all architectures and data scenarios, Adam converged faster than RAdam but slower than Lookahead. Its adaptive learning rates and momentum-based updates ensured steady optimization, but without the variance control of RAdam or the path smoothing of Lookahead, it lacked standout performance in either convergence or accuracy. Still, Adam's balance of simplicity, speed, and generalization makes it a dependable choice, especially when resources are limited, and extreme data noise is absent.

Adagrad experienced fast early convergence, but misleading stability. Interestingly, Adagrad often converged quickly on paper—sometimes faster than Adam or even Lookahead in CNN–LSTM—but this convergence was misleading. Its performance in terms of forecasting accuracy is the weakest across all optimizers, with MSE values consistently higher, particularly in Dataset 2. This is due to Adagrad's aggressive learning rate decay, which causes rapid early convergence but leads to stagnation before meaningful learning is complete. Thus, while Adagrad might reach convergence thresholds in fewer epochs, it often does so prematurely, compromising model performance for training speed.

5.6 Computational footprint and edge feasibility

Because the stated motivation is real-time deployment on plant-side or micro-grid edge devices, we benchmarked the computational cost of each trained model on the same hardware: a free Google Colab instance (2 × Intel - Xeon @ 2.20 GHz CPU, 12.7 GB RAM, no GPU). Table 17 presents the average per-sample inference latency and peak memory footprint for each network architecture; these figures are optimizer-agnostic and therefore apply equally to Adam, Adagrad, RAdam, and Lookahead.

A single 60 min forecast requires at most 0.44 ms and 11.53 MB of memory, comfortably below the 10 ms/50 MB budget typical of ARM-based edge controllers. Even the slowest optimizer (RAdam) converged within 53 epochs (Table 16), corresponding to ≈19 min of wall-clock training on the same CPU. These measurements confirm that all evaluated models satisfy the latency and memory constraints previously cited for on-device PV forecasting.

5.7 Discussion

The performance gaps observed in Sects. 5.1–5.4 can be traced back to how each optimizer modulates its learning rate in response to gradient statistics.

RAdam's variance rectification: RAdam augments Adam with an on-the-fly estimate of the variance of the adaptive learning rate itself. During the first few hundred updates its step size is deliberately under-corrected until a reliable running variance is available; after that, the algorithm behaves like Adam but with per-dimension steps

Table 17 Average training and inference cost (CPU only, Dataset 1, 60 min horizon)

Architecture	Training time [s/epoch]	Inference latency [ms/sample]	Mem [MB]
LSTM	17	0.37	11.53
CNN–LSTM	12	0.32	9.24
LSTM–Autoencoder	21	0.44	10.29

"Mem" = peak resident memory during forward pass. Values are averaged over 5 seeds; s.d. <4 % in all cases

pre-scaled by $\sqrt{\widehat{\mathrm{Var}}[g]}$. For the noisy one-minute PV series (high-variance gradients) this slow-start mechanism prevents early overshooting, while the subsequent per-dimension scaling lets the model follow steep directions (e.g., cloud-edge ramps) without collapsing the step size in flatter regions (e.g., clear-sky plateaus). Hence RAdam retains large enough updates to keep learning when sampling intervals grow, data become sparse, or long blocks are missing–explaining its dominant MSE and R^2 scores.

Lookahead's path smoothing: Lookahead performs k "fast" inner-loop steps with a base optimizer (Adam in our case) and then replaces the model weights with an α-interpolated average of the fast and slow weights. The interpolation acts as a low-pass filter on the optimization path, damping the exploding gradient updates typical of convolutional front ends. This yields the superior stability seen in the CNN–LSTM results (especially under coarse sampling and mild missingness), where local spatial features can cause large, correlated gradients that benefit from smoothing.

Adagrad's aggressive decay: Adagrad's cumulative sum of squared gradients, $\sum_t g_t^2$, enters the denominator of its learning-rate schedule; in our data this sum grows rapidly because irradiance-driven gradients have persistent, large-magnitude components. Consequently the effective step size dwindles after a few thousand batches, producing premature "convergence" with the highest residual error and the lowest R^2.

Adam's balance but non-rectification: Adam uses exponential moving averages of the first and second moments but lacks RAdam's variance rectification. It therefore starts with larger steps than RAdam (faster initial convergence) yet inherits Adam's tendency to over-shoot under highly stochastic gradients, explaining the middling MSE/R^2 and the moderate convergence speed measured in Tables 15 and 16.

Therefore, optimizers that adaptively modulate their learning rates in a variance-aware (RAdam) or path-smoothing (Lookahead) manner are better equipped to handle the non-stationary, intermittently sparse gradients endemic to short-horizon PV forecasting.

5.8 Limitations and practical trade-offs

While this study presents a comprehensive comparison of optimizer performance for short-term solar PV forecasting, several limitations should be acknowledged to contextualize the findings and guide future research. First, all neural networks were trained using manually tuned hyperparameters. Although this approach ensured consistency across experiments and reduced computational burden, automated methods such as Bayesian optimization or grid search may yield more optimized configurations for specific model–optimizer pairings.

Second, although two geographically and climatically diverse datasets were used to test generalizability, both originate from the same source (NREL PVDAQ). Broader validation across different continents, seasonal patterns, or urban microclimates would enhance the applicability of the conclusions and remains an important avenue for future work.

Third, the analysis assumes clean preprocessing and does not account for real-time data ingestion or live deployment issues such as latency, sensor noise, or dynamic retraining. Future work should explore these aspects in edge or streaming environments.

Additionally, the forecasting horizon was fixed at 60 min, and the models were not tested for longer-term predictions or multi-step horizons, which are increasingly relevant for storage planning and dispatch optimization. Nevertheless, this study targets the 60-min horizon that is most critical for intra-hour PV dispatch, the proposed optimizer ranking is not inherently tied to single-step prediction. All three backbones can be converted to multi-step forecasting by (i) feeding them a horizon-length output layer, or (ii) rolling the network auto-regressively. Previous work shows that LSTM-based models retain skill out to 6 h when trained in this manner, while CNN–LSTM hybrids can scale to 24 h day-ahead forecasts with minor architectural changes [36]. Future work will therefore extend the benchmark to longer horizons in order to verify whether RAdam and Lookahead still dominate when long-range error accumulation and weather-model uncertainty become significant.

Table 18 Summary of optimizer behaviors based on convergence time and forecasting accuracy

Optimizer	Convergence time	Forecast accuracy	Strengths	Limitations
Lookahead	Fastest	High (best in CNN–LSTM)	Smooth and stable convergence; effective in deep hybrid models	Less robust under block-wise missing data
RAdam	Slowest	**Highest**	Very stable under noisy and incomplete data; excellent generalization	Long training time; delayed convergence
Adam	Moderate	Moderate	Balanced choice; steady updates across models	Outperformed by RAdam and Lookahead in most settings
Adagrad	Appears fast (but premature)	**Lowest**	Simple and fast initial training; handles sparse gradients well	Early stagnation; poor learning in volatile data

Values in bold mark the top-performing model–optimizer pair within each scenario

From a deployment perspective, a trade-off emerges between convergence time and forecasting accuracy. RAdam consistently yields the most robust predictions but at the cost of slower convergence. In contrast, Lookahead enables faster convergence, especially in deeper architectures like CNN-LSTM, but may sacrifice some robustness under noisy or incomplete data. Thus, the choice of optimizer should be driven by application-specific constraints, such as training time availability, compute limitations, and acceptable forecasting error margins.

6 Conclusion

This study benchmarked four optimizers—Adam, Adagrad, RAdam, and Lookahead—across three neural network architectures for short-term solar PV power forecasting using two real-world datasets characterized by diverse climatic and data conditions. Results highlighted that RAdam consistently provided the highest forecasting accuracy, particularly in LSTM, reducing forecast error by approximately 32% compared to Adam and 54% compared to Adagrad in challenging scenarios with 30% data loss. Even under cleaner conditions with only 5% data loss, RAdam maintained an accuracy advantage of roughly 9% over Adam and 48% over Adagrad. However, these accuracy gains came at the expense of slower convergence rates. Conversely, Lookahead achieved notably faster convergence, outperforming RAdam by approximately 22–40% and Adam by about 5–15%, particularly in deep hybrid models like CNN–LSTM. Adam exhibited balanced but generally intermediate performance, whereas Adagrad, despite rapid early convergence, consistently demonstrated poor generalization due to aggressive learning rate decay. Table 18 summarizes each optimizer's comparative strengths, weaknesses, and convergence behaviors.

The forecasting improvements demonstrated in this study have clear practical implications for integrating PV generation into power grids. Specifically, the observed reductions in forecast errors enable power system operators to maintain tighter reserve margins. This means grid operators can more confidently predict PV power availability and thus reduce the amount of backup or spinning reserves traditionally held in anticipation of forecasting errors. In practical terms, smaller reserve margins directly lead to lower operational costs and more efficient grid operation. Additionally, enhanced forecasting accuracy contributes to smoother inverter set-points, allowing for improved control strategies that minimize voltage fluctuations and frequency deviations. Accurate forecasts facilitate proactive inverter management, helping to mitigate rapid changes in power output due to intermittency and variability of solar resources. This directly enhances the stability and reliability of power delivery from solar installations, contributing positively to grid resilience and operational efficiency.

Overall, these optimizer-driven improvements not only enhance forecast accuracy from a statistical perspective

but also advance the practical state-of-the-art in PV forecasting pipelines, providing tangible benefits to power system planning, operational stability, and integration of renewable energy sources into existing grids.

This study lays the groundwork for future efforts that could extend both the depth and breadth of optimizer evaluation in solar PV forecasting tasks. First, integrating additional input variables such as temperature, wind speed, or cloud cover could enhance model input richness, particularly in multivariate forecasting. Second, extending the study to include long-term and multi-step forecasting horizons would be valuable for storage and dispatch planning. Finally, deploying these models in edge-computing scenarios with real-time data streams would offer practical insights into inference latency, model update strategies, and live fault handling under operational constraints.

Author contributions Conceptualization, SD, GG and GSG; Methodology, SD, GG and GSG; Software, SD; Validation, SD; Formal analysis, SD, GG and GSG; Investigation, SD, GG and GSG; Writing—original draft preparation, SD, GG and GSG; Writing—Review & editing, SD, GG and GSG; Supervision, GSG.

Funding Open access funding provided by Politecnico di Milano within the CRUI-CARE Agreement. Open access funding provided by Politecnico di Milano within the CRUI-CARE Agreement

Data availability All data used are available from public repositories cited in the manuscript.

Materials availability Not applicable.

Code availability Not applicable.

Declarations

Conflict of interest All authors certify that they have no affiliations with or involvement in any organization or entity with any financial interest or non-financial interest in the subject matter or materials discussed in this manuscript.

Consent for publication All authors consent to publication of the manuscript if accepted.

Ethical approval All authors declare to comply with the ethical standards required by this journal.

Open Access This article is licensed under a Creative Commons Attribution 4.0 International License, which permits use, sharing, adaptation, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if changes were made. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit http://creativecommons.org/licenses/by/4.0/.

References

1. Alraddadi M, Conejo AJ, Lima RM (2021) Expansion planning for renewable integration in power system of regions with very high solar irradiation. J Mod Power Syst Clean Energy 9:485–494. https://doi.org/10.35833/MPCE.2019.000112
2. Shafiullah M, Ahmed SD, Al-Sulaiman FA (2022) Grid integration challenges and solution strategies for solar PV systems: a review. IEEE Access 10:52233–52257. https://doi.org/10.1109/ACCESS.2022.3174555
3. Hong T, Pinson P, Wang Y, Weron R, Yang D, Zareipour H (2020) Energy forecasting: a review and outlook. IEEE Open Access J Power Energy 7:376–388. https://doi.org/10.1109/OAJPE.2020.3029979
4. Dhingra S, Gruosso G, Gajani GS (2023) Solar PV power forecasting and ageing evaluation using machine learning techniques. In: IECON 2023-49th annual conference of the IEEE industrial electronics Society, pp 1–6. https://doi.org/10.1109/IECON51785.2023.10312446
5. Wentz VH, Maciel JN, Gimenez Ledesma JJ, Ando Junior OH (2022) Solar irradiance forecasting to short-term PV power: accuracy comparison of ANN and LSTM models. Energies. https://doi.org/10.3390/en15072457

6. Salman D, Direkoglu C, Kusaf MEA (2024) Hybrid deep learning models for time series forecasting of solar power. Neural Comput Appl 36:9095–9112. https://doi.org/10.1007/s00521-024-09558-5
7. Symeonidis C, Nikolaidis N (2025) Efficient deterministic renewable energy forecasting guided by multiple-location weather data. Neural Comput Appl. https://doi.org/10.1007/s00521-024-10607-2
8. Ahmed R, Sreeram V, Mishra Y, Arif MD (2020) A review and evaluation of the state-of-the-art in PV solar power forecasting: techniques and optimization. Renew Sustain Energy Rev. https://doi.org/10.1016/j.rser.2020.109792
9. Kingma DP, Ba J (2014) Adam: a method for stochastic optimization. International conference on learning representations (ICLR). https://doi.org/10.48550/arXiv.1412.6980
10. Duchi J, Hazan E, Singer Y (2011) Adaptive subgradient methods for online learning and stochastic optimization. J Mach Learn Res 12:2121–2159
11. Ruder S (2017) An overview of gradient descent optimization algorithms. arXiv preprint https://doi.org/10.48550/arXiv.1609.04747
12. Zeiler M (2012) Adadelta: an adaptive learning rate method. arXiv preprint https://doi.org/10.48550/arXiv.1212.5701
13. Zhang MR, Lucas J, Hinton G, Ba J (2019) Lookahead optimizer: k steps forward, 1 step back. In: Proceedings of the 33rd international conference on neural information processing systems 861:12. https://doi.org/10.48550/arXiv.1907.08610
14. Liu L, Jiang H, He P, Chen W, Liu X, Gao J, Han J (2019) On the variance of the adaptive learning rate and beyond. arXiv preprint https://doi.org/10.48550/arXiv.1908.03265
15. Zhuang J, Tang T, Ding Y, Tatikonda S, Dvornek N, Papademetris X, Duncan JS (2020) Adabelief optimizer: adapting stepsizes by the belief in observed gradients. arXiv preprint https://doi.org/10.48550/arXiv.2010.07468
16. Park J, Lee S, Song B (2023) Stable quantization-aware training with adaptive gradient clipping. In: 2023 International conference on electronics, information, and communication (ICEIC), pp 1–3. https://doi.org/10.1109/ICEIC57457.2023.10049939
17. Zaeed M, Islam TZ, Indic V (2024) Characterize and compare the performance of deep learning optimizers in recurrent neural network architectures. In: 2024 IEEE 48th annual computers, software, and applications conference (COMPSAC), pp 39–44. https://doi.org/10.1109/COMPSAC61105.2024.00016
18. Suresha HS, Parthasarathy SS (2020) Alzheimer disease detection based on deep neural network with rectified Adam optimization technique using MRI analysis. In: 2020 Third international conference on advances in electronics, computers and communications (ICAECC), pp 1–6. https://doi.org/10.1109/ICAECC50550.2020.9339504
19. Shi H, Wang L, Scherer R, Wozniak M, Zhang P, Wei W (2021) Short-term load forecasting based on adabelief optimized temporal convolutional network and gated recurrent unit hybrid neural network. IEEE Access 9:66965–66981. https://doi.org/10.1109/ACCESS.2021.3076313
20. Dhingra S, Gruosso G, Gajani GS (2023) Modelling ageing and power production of solar PV using machine learning techniques. In: 2023 International conference on electrical, computer and energy technologies (ICECET), pp 1–6. https://doi.org/10.1109/ICECET58911.2023.10389351
21. Sharma J, Soni S, Paliwal P, Saboor S, Chaurasiya PK, Sharifpur M, Khalilpoor N, Afzal A (2022) A novel long term solar photovoltaic power forecasting approach using LSTM with Nadam optimizer: a case study of india. Energy Sci Eng 10:2909–2929. https://doi.org/10.1002/ese3.1178
22. Bhatnagar M, Dwivedi V, Rozinaj G (2023) Short-term electric load forecast model using the combination of ant lion optimization with bi-LSTM network. In: 2023 30th International conference on systems, signals and image processing (IWSSIP), pp 1–5. https://doi.org/10.1109/IWSSIP58668.2023.10180242
23. Cai H, Chen X, Ling J, Xu Q (2022) Short-term load forecasting based on Radam optimized CNN-BiLSTM-AE hybrid model. In: 2022 Power system and green energy conference (PSGEC), pp 626–631. https://doi.org/10.1109/PSGEC54663.2022.9881115
24. Jiang F, He J, Peng Z (2018) Short-term wind power forecasting based on BP neural network with improved ant lion optimizer. In: 2018 37th Chinese control conference (CCC), pp 8543–8547. https://doi.org/10.23919/ChiCC.2018.8482950
25. Sulochana BC, Pragada B, Kiran BC, Reddy G, KM (2024) Machine learning in weather forecasting: a comparative approach with emphasis on neural networks and optimizer. In: 2024 International conference on advances in modern age technologies for health and engineering science (AMATHE), pp 1–7. https://doi.org/10.1109/AMATHE61652.2024.10582220
26. BM, PM, K A, SD (2022) Early prediction of health disease for long–short term memory using AdaGrad model. In: 2022 IEEE 2nd Mysore sub section international conference (MysuruCon), pp 1–6. https://doi.org/10.1109/MysuruCon55714.2022.9972556
27. Deline C, Perry K, Muller M, Sekulic W, Jordan D Photovoltaic Data Acquisition (PVDAQ) Public Datasets. https://data.openei.org/submissions/4568 (accessed on 3 Jan 2024)
28. Jalali SMJ, Ahmadian S, Kavousi-Fard A, Khosravi A, Nahavandi S (2022) Automated deep CNN-LSTM architecture design for solar irradiance forecasting. IEEE Trans Syst Man Cybern Syst 52(1):54–65. https://doi.org/10.1109/TSMC.2021.3093519

29. Zivkovic M, Jovanovic L, Pavlov M, Bacanin N, Dobrojevic M, Salb M (2023) Optimized recurrent neural networks with attention for wind farm energy generation forecasting. In: 2023 16th International conference on advanced technologies, systems and services in telecommunications (TELSIKS), pp 187–190. https://doi.org/10.1109/TELSIKS57806.2023.10316047
30. Zhang Z, Wang Y, Peng JEA (2024) An improved self-attention for long-sequence time-series data forecasting with missing values. Neural Comput Appl 36:3921–3940. https://doi.org/10.1007/s00521-023-09347-6
31. Srivastava A, Rawat BS, Singh G, Bhatnagar V, Saini PK, Dhondiyal SA (2023) A review of optimization algorithms for training neural networks. In: 2023 International conference on sustainable emerging innovations in engineering and technology (ICSEIET), pp 886–890. https://doi.org/10.1109/ICSEIET58677.2023.10303287
32. Llugsi R, Yacoubi S, Fontaine A, Lupera P (2021) Comparison between Adam, AdaMax and Adam W optimizers to implement a weather forecast based on neural networks for the Andean city of Quito. In: 2021 IEEE fifth ecuador technical chapters meeting (ETCM), pp 1–6. https://doi.org/10.1109/ETCM53643.2021.9590681
33. Ward R, Wu X, Bottou L (2021) Adagrad stepsizes: sharp convergence over nonconvex landscapes. arXiv preprint https://doi.org/10.48550/arXiv.1806.01811
34. Fang J, Fong C, Yang P, Hung C, Lu W, Chang C (2020) Adagrad gradient descent method for AI image management. In: 2020 IEEE international conference on consumer electronics-Taiwan (ICCE-Taiwan), pp 1–2. https://doi.org/10.1109/ICCE-Taiwan49838.2020.9258085
35. Zhou P, Yan H, Yuan X, Feng J, Yan S (2024) Towards understanding why lookahead generalizes better than SGD and beyond. In: Proceedings of the 35th international conference on neural information processing systems. NIPS '21, pp 27290–27304
36. Qu J, Qian Z, Pei Y (2021) Day-ahead hourly photovoltaic power forecasting using attention-based CNN-LSTM neural network embedded with multiple relevant and target variables prediction pattern. Energy. https://doi.org/10.1016/j.energy.2021.120996

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Authors and Affiliations

Saloni Dhingra[1] · Giambattista Gruosso[1] · Giancarlo Storti Gajani[1]

✉ Saloni Dhingra
saloni.dhingra@polimi.it

Giambattista Gruosso
giambattista.gruosso@polimi.it

Giancarlo Storti Gajani
giancarlo.stortigajani@polimi.it

[1] DEIB, Politecnico di Milano, P.zza L. da Vinci 32, Milano 20133, MI, Italy